# PROGRAMING FUNDAMENTALS

## FINAL LAB REPORT (WEEK 1 – 15)


**NAME:** Soban Haider

**REG NUMBER:** 2026(S)-CYS-103

**PROGRAM:** Cyber Security

**SECTION:** B

**SESSION:** Spring 2026

**TEACHER:** Hafiz Muhammad Mubashar Imran

| University Of Engineering and Technology, Lahore<br>Computer Engineering Department |
|---|

| | |
|---|---|
| **Course Name: Programing Fundamentals** | **Course Code:** CMPE-112L |
| **Assignment Type:** | **Dated: 23 March,2026** |
| **Semester: 1st** | **Session: Spring 2026** |
| **Lab/Project/Assignment #:** 1 | **CLOs to be covered: CLO 1** |
| **Lab Title: Input/Output, Data Types, Operators, Conditional Statements(if/else)** | **Teacher Name:** Hafiz Muhammad Mubashar |

## Lab Evaluation:

| **CLO1** | Apply appropriate programming techniques to create executable programs to solve well defined problems | | | | | |
|---|---|---|---|---|---|---|
| | **Level1** | **Level2** | **Level3** | **Level4** | **Level5** | **Level6** |
| (5) | | | | | | |
| | | | | | **Total** | /5 |

## Rubrics for Current Lab Evaluation

| Scale | Marks | Level | Rubric |
|---|---|---|---|
| Excellent | 5 | L1 | Submitted all lab tasks, BONUS task, have good understanding. |
| Very Good | 4 | L2 | Submitted the lab tasks but have good understanding |

| | | | |
|---|---|---|---|
| Good | 3 | L3 | Submitted the lab tasks but have weak understanding. |
| Basic | 2 | L4 | Submitted the lab tasks but have no understanding. |
| Barely Acceptable | 1 | L5 | Submitted only one lab task. |
| Not Acceptable | 0 | L6 | Did not attempt |

## LAB NO 1

**Lab Goals/Objectives:**

By reading this manual, students will be able to:

- Learn about Input and Output commands in Python.
- Understand different data types used in Python.
- Use operators to perform various operations on data.
- Implement decision-making using Conditional Statements (`if`, `elif`, and `else`).

## Equipment Required:

Computer system with PyCharm IDE and Python package installed on it.

## Theory:

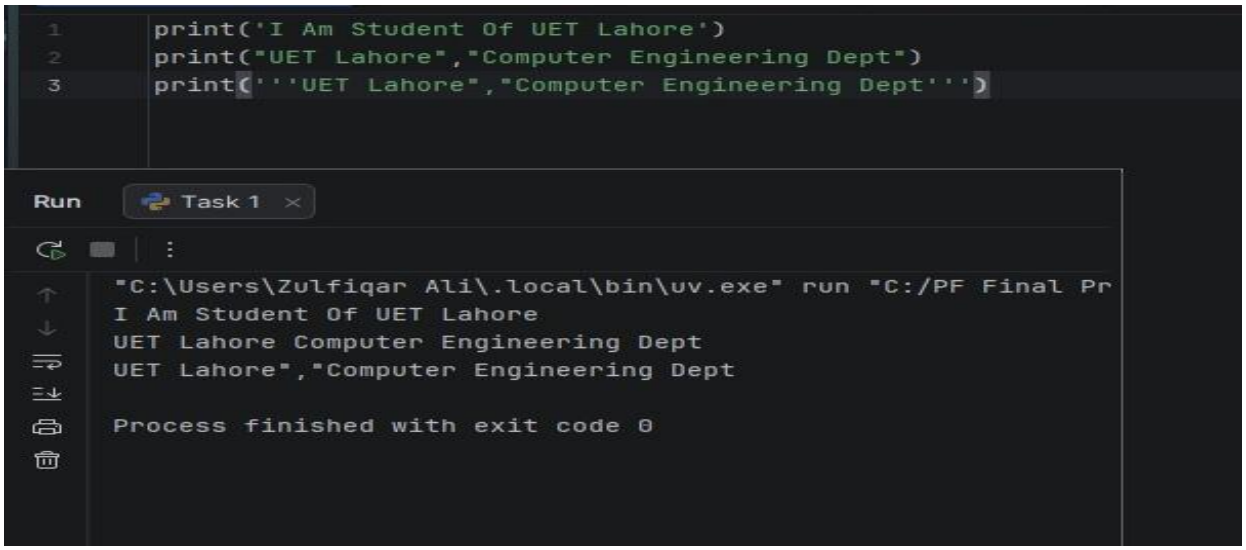
Python provides the `input()` function to receive data from the user and the `print()` function to display output on the screen. It supports various data types such as integers (`int`), floating-point numbers (`float`), strings (`str`), and Boolean values (`bool`) for storing different kinds of information. Operators are used to perform arithmetic, comparison, logical, and assignment operations on data. Conditional statements (`if`, `elif`, and `else`) allow programs to make decisions and execute different blocks of code based on specified conditions.

**Lab Work:**

**Task 1:**

Write a Python program to display text on the screen using different string formats and the `print()` function.

**Result:**




**Task 2:**

Write a Python program to demonstrate the use of escape sequences (`\n`, `\t`, and `\\`) for formatting text output.

**Result:**

```
print("I AM Student\nOf UET")
print("First\tSecond\tThird")
print("This is a backslash: \\")
```

```
Run  Task 2
"C:\Users\Zulfiqar Ali\.local\bin\uv.exe" run "C:/PF Final Pr
I AM Student
Of UET
First   Second  Third
This is a backslash: \

Process finished with exit code 0
```

**Task 3:**

Write a Python program to declare an integer variable, display its value, and determine its data type using the `type()` function.

**Result:**

```
age = 20

print("Age =", age)
print(type(age))
```

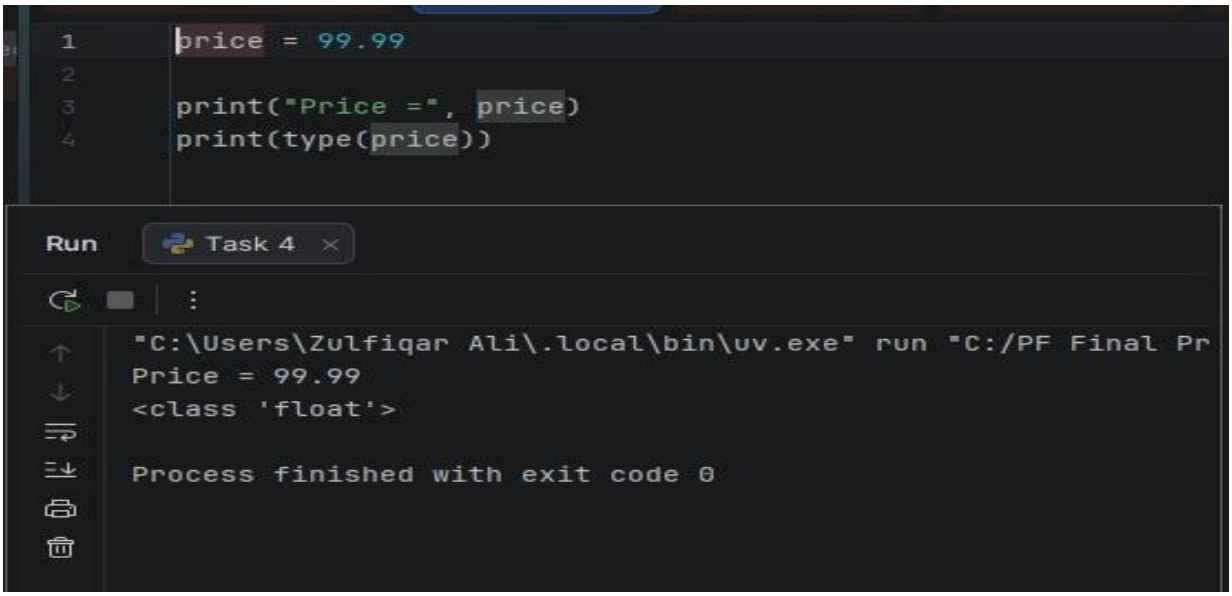
```
Run  Task 3
"C:\Users\Zulfiqar Ali\.local\bin\uv.exe" run "C:/PF Final
Age = 20
<class 'int'>

Process finished with exit code 0
```

**Task 4:**

Write a Python program to declare a floating-point variable, display its value, and identify its data type using the `type()` function.

**Result:**




## Task 5:

Write a Python program to declare a Boolean variable, display its value, and determine its data type using the `type()` function.

**Result:**


**Task 6:**

Write a Python program to demonstrate the use of Boolean data type and display its class type using the `type()` function.

**Result:**

```python
passed = True

print("Passed =", passed)
print(type(passed))
```

```
Run  Task 6
"C:\Users\Zulfiqar Ali\.local\bin\uv.exe" run "C:/PF Final Pr
Passed = True
<class 'bool'>

Process finished with exit code 0
```

**Task 7:** Write a Python program to compare two numbers using relational operators (==, !=, >, and <) and display the results.

**Result:**

```python
a = 15
b = 20

print("a == b :", a == b)
print("a != b :", a != b)
print("a > b :", a > b)
print("a < b :", a < b)
```

```
Run  Task 7
"C:\Users\Zulfiqar Ali\.local\bin\uv.exe" run "C:/PF Final Pr
a == b : False
a != b : True
a > b : False
a < b : True

Process finished with exit code 0
```

**Task 8:**

Write a Python program to perform logical operations using the logical operators `and`, `or`, and `not`.

**Result:**


## Task 9:

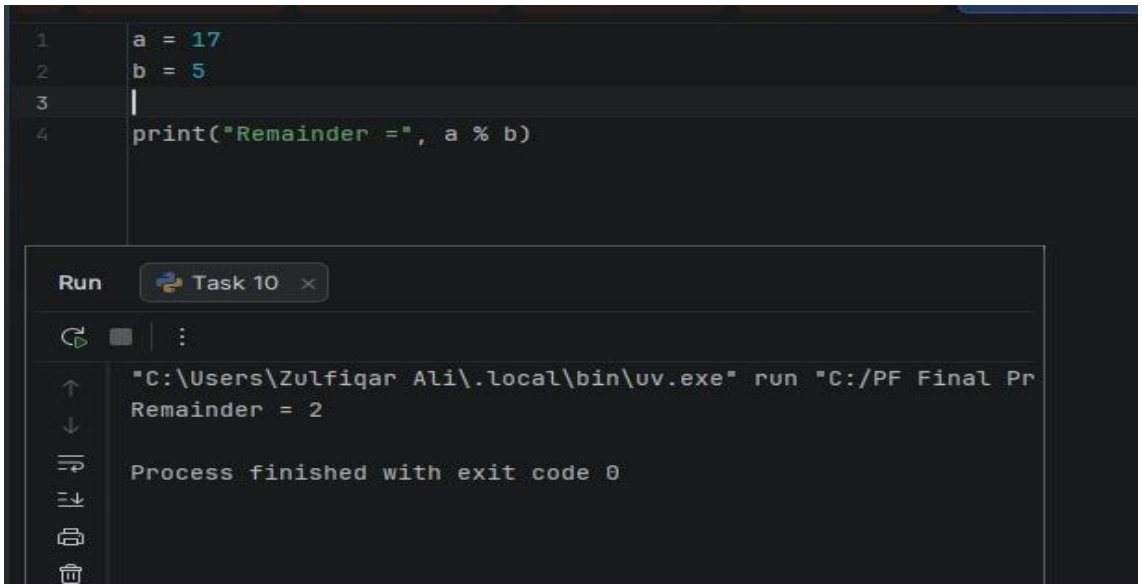
Write a Python program to demonstrate the use of assignment operators (+=, −=, and *=) and display the updated values.

**Result:**


**Task 10:**

Write a Python program to calculate and display the remainder obtained after dividing one number by another using the modulus (%) operator.

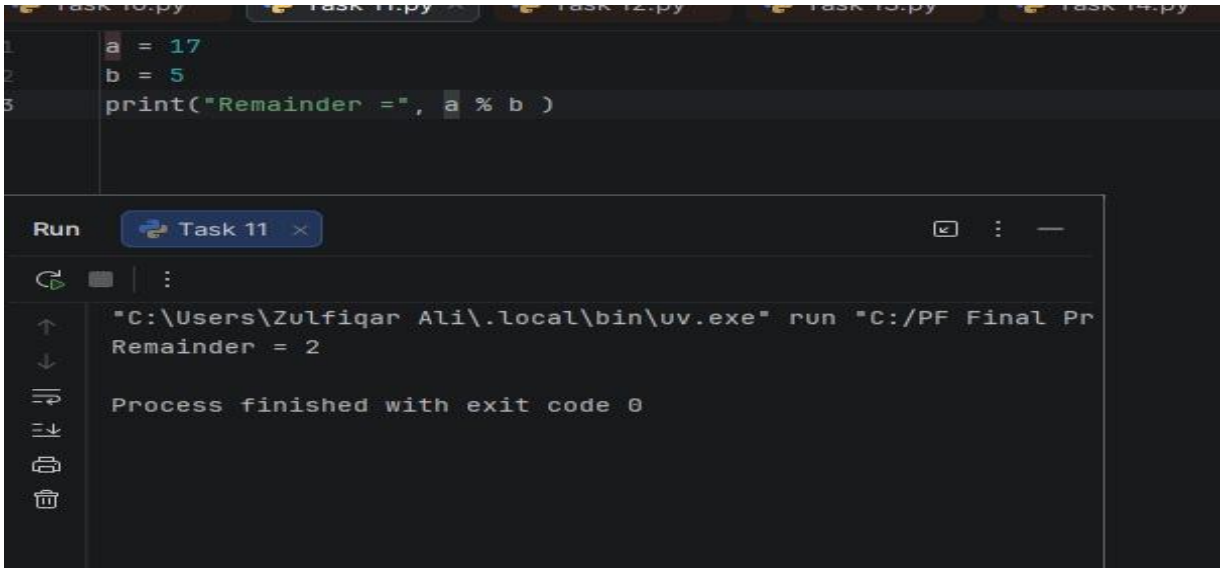
**Result:**



**Task 11:**

Write a Python program to calculate the power of a number using the exponentiation (**) operator and display the result.

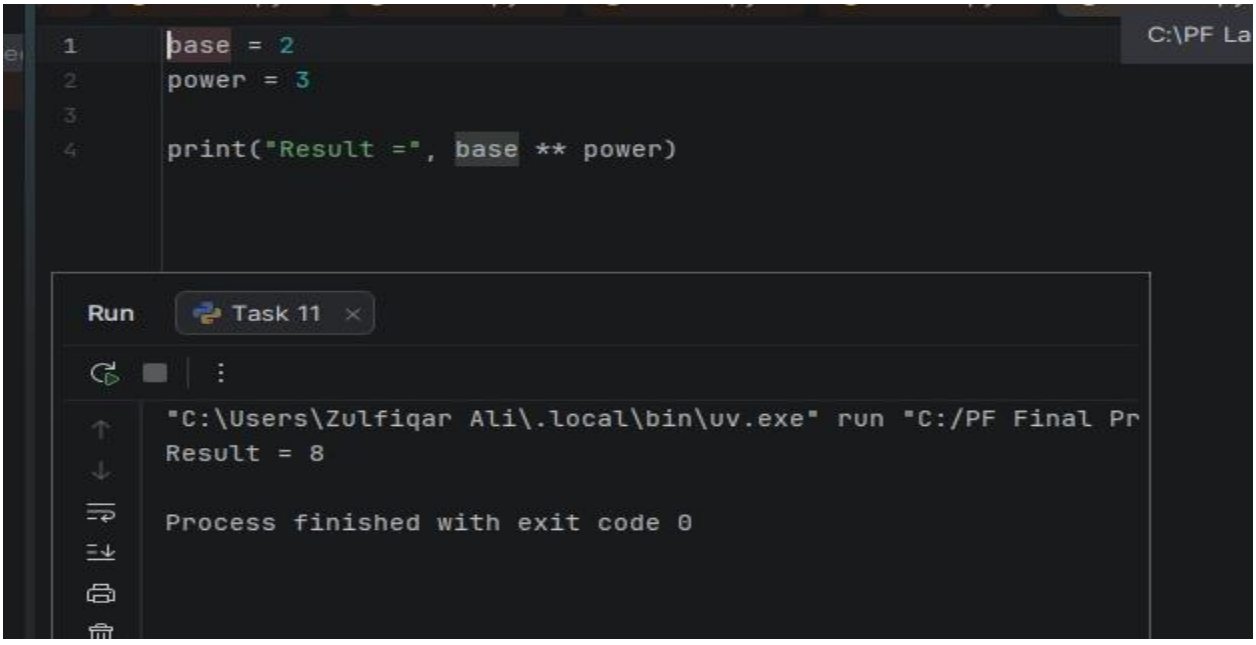
**Result:**




**Task 12:**

Write a Python program to demonstrate arithmetic operations and display the result of mathematical calculations.

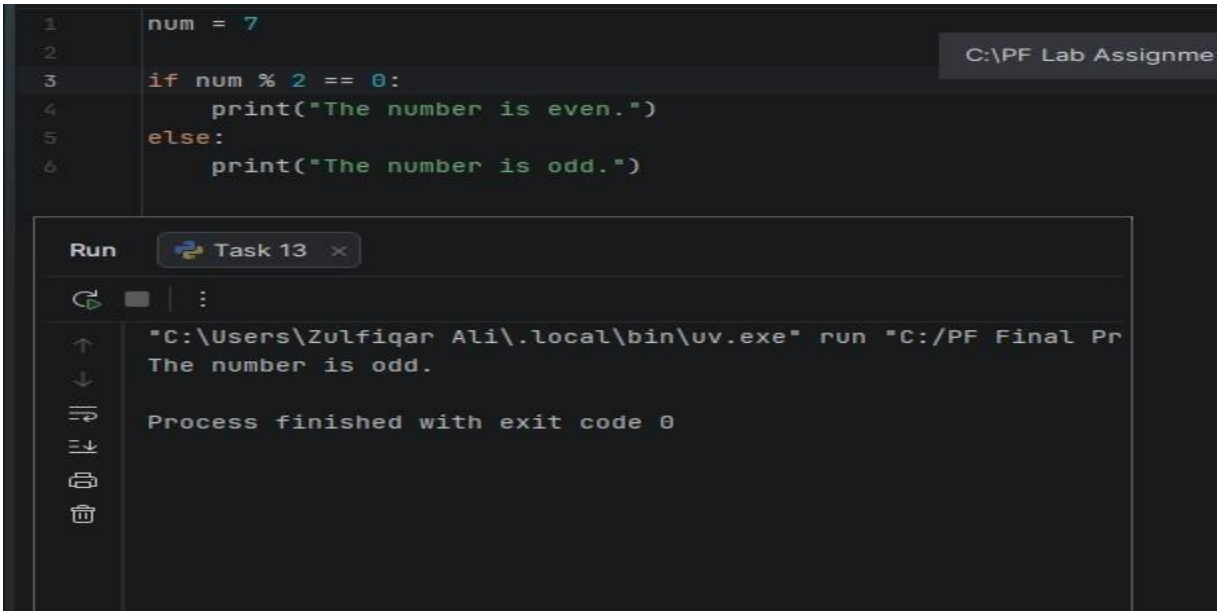
**Result:**




**Task 13:**

Write a Python program that uses an `if-else` statement to determine whether a given number is even or odd.

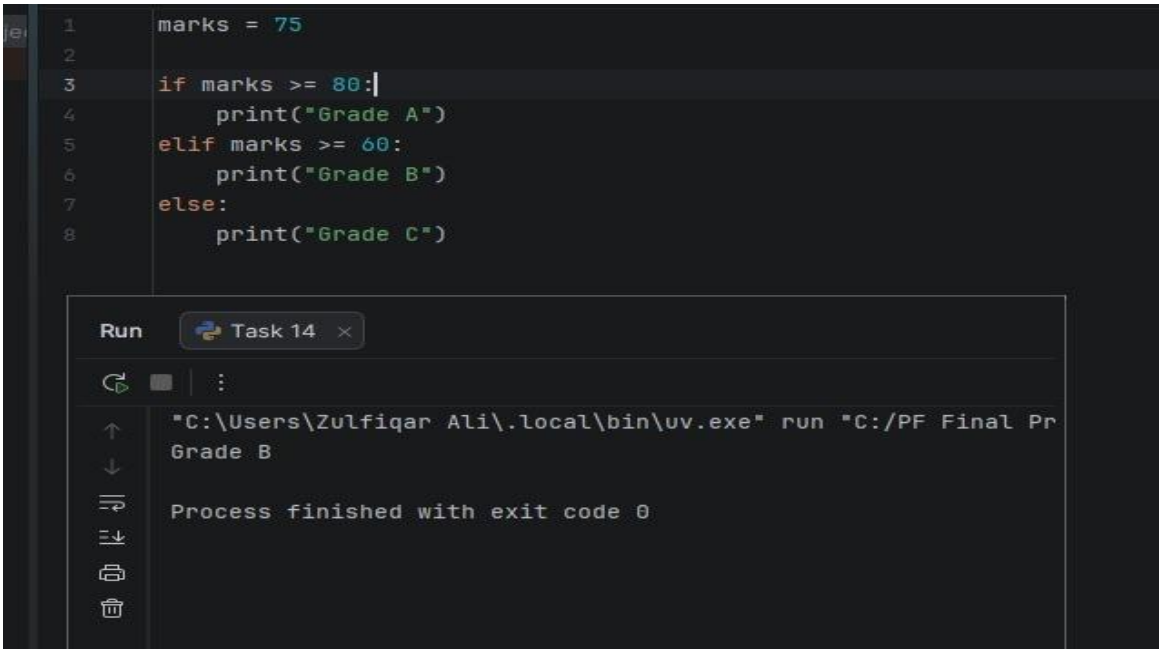
**Result:**




**Task 14:**

Write a Python program that uses `if-elif-else` statements to assign grades based on the marks obtained by a student.

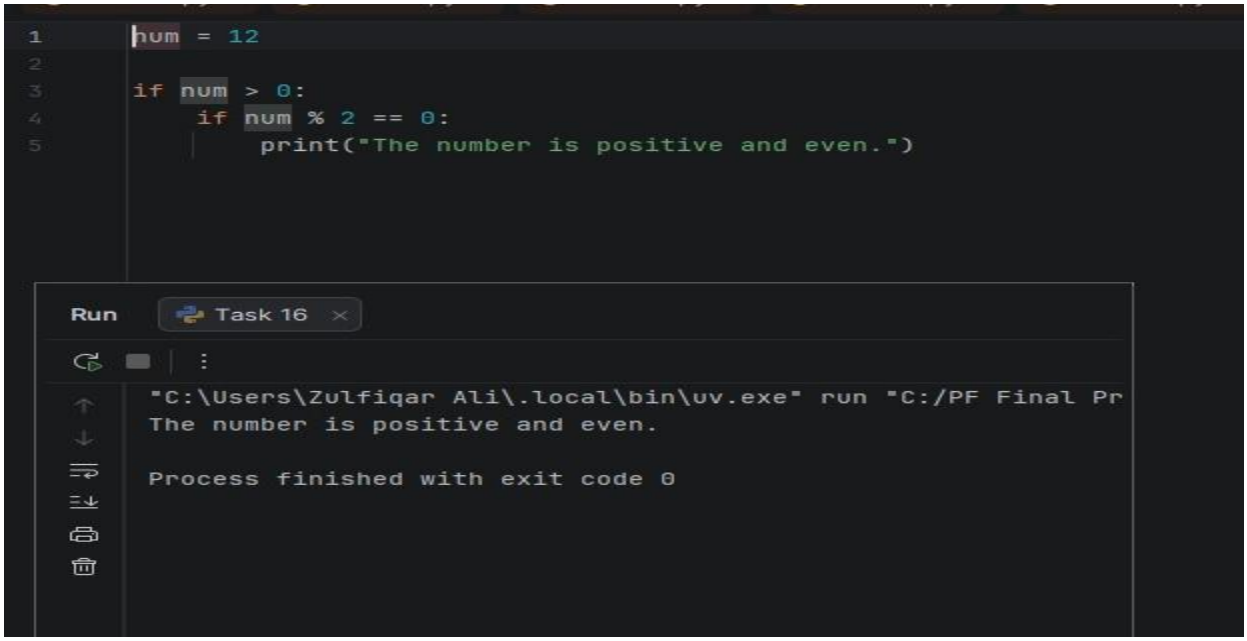
**Result:**




## Task 16:

Write a Python program that checks whether a number is positive and even using nested `if` statements.

**Result:**




**Conclusions:**

In this lab, we learned how to use input/output functions, data types, operators, conditional statements (`if`, `elif`, `else`), and `for` loops in Python. These concepts help in creating simple and efficient programs and form the foundation of Python programming.

| University Of Engineering and Technology, Lahore<br>Computer Engineering Department |
|---|

| | |
|---|---|
| **Course Name: Programing Fundamentals** | **Course Code:** CMPE-112L |
| **Assignment Type:** | **Dated: 2 March,2026** |
| **Semester: 1st** | **Session: 2026 Spring** |
| **Lab/Project/Assignment #: 2** | **CLOs to be covered: CLO 1** |
| **Lab Title: For Loops** | **Teacher Name:** Hafiz Muhammad Mubashar |

**Lab Evaluation:**

| **CLO1** | Apply appropriate programming techniques to create executable programs to solve well defined problems | | | | | |
|---|---|---|---|---|---|---|
| | **Level1** | **Level2** | **Level3** | **Level4** | **Level5** | **Level6** |
| (5) | | | | | | |
| | | | | | **Total** | /5 |

**Rubrics for Current Lab Evaluation**

| Scale | Marks | Level | Rubric |
|---|---|---|---|
| Excellent | 5 | L1 | Submitted all lab tasks, BONUS task, have good understanding. |
| Very Good | 4 | L2 | Submitted the lab tasks but have good understanding |

| | | | |
|---|---|---|---|
| Good | 3 | L3 | Submitted the lab tasks but have weak understanding. |
| Basic | 2 | L4 | Submitted the lab tasks but have no understanding. |
| Barely Acceptable | 1 | L5 | Submitted only one lab task. |
| Not Acceptable | 0 | L6 | Did not attempt |

# LAB NO 2

## Lab Goals/Objectives:

By reading this manual, students will be able to:

- Understand the concept of repetition in programming.
- Learn how to use `for` loops in Python.
- Iterate through a sequence of values using the `range()` function.
- Develop programs that perform repetitive tasks efficiently.

## Equipment Required:

Computer system with PyCharm IDE and Python package installed on it.

## Theory:

A `for` loop is used in Python to execute a block of code repeatedly. It iterates over a sequence such as a range of numbers, a string, or a list. The `for` loop helps reduce code repetition and makes programs more efficient and easier to understand.

## Lab Work:

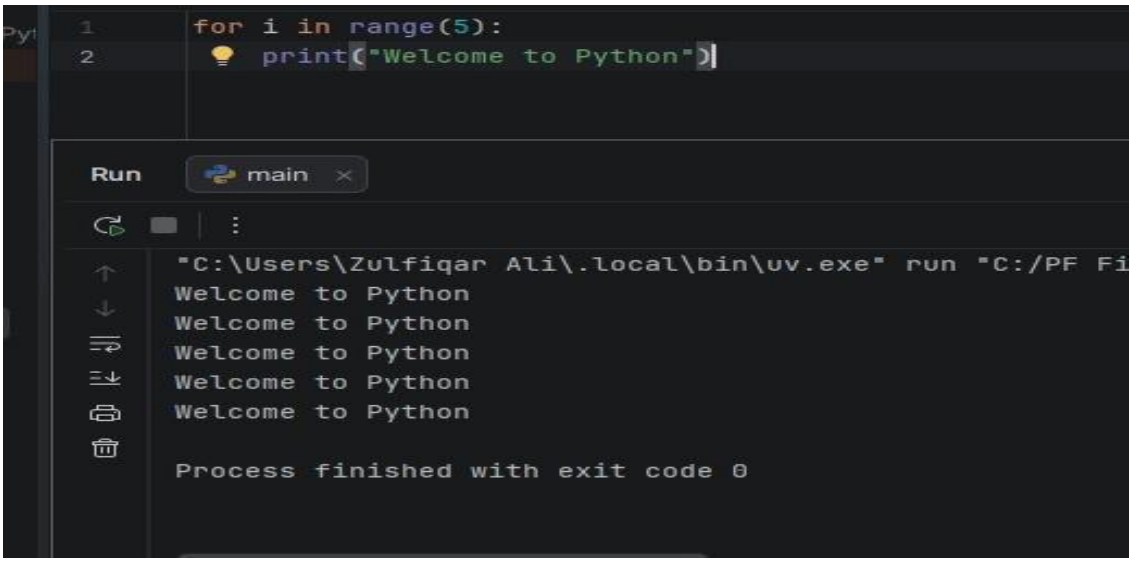
### Task 1:

Write a Python program that uses a `for` loop to display numbers from 1 to 5.

**Result:**


**Task 2:**

Write a Python program that uses a `for` loop to print a message five times.




**Result**

**Task 3:**

Write a Python program that uses a `for` loop to display each character of a string.

**Result:**

```
name = "Python"

for ch in name:
    print(ch)
```

```
Run  Task 1
"C:\Users\Zulfiqar Ali\.local\bin\uv.exe" run "C:/PF Final
P
y
t
h
o
n

Process finished with exit code 0
```

**Task 4:**

Write a Python program that uses a `for` loop to calculate the sum of numbers from 1 to 10.

**Result:**

```
total = 0

for i in range(1, 11):
    total += i

print("Sum =", total)
```

```
Run  Task 3
"C:\Users\Zulfiqar Ali\.local\bin\uv.exe" run "C
Sum = 55

Process finished with exit code 0
```

.

**Conclusions:** In this lab, we learned how to use `for` loops in Python to repeat tasks, iterate through sequences, and perform calculations efficiently

| University Of Engineering and Technology, Lahore<br>Computer Engineering Department |
|---|

| | |
|---|---|
| **Course Name: Programing Fundamentals** | **Course Code:** CMPE-112L |
| **Assignment Type:** | **Dated: 16 March,2026** |
| **Semester: 1st** | **Session: 2026 Spring** |
| **Lab/Project/Assignment #: 3** | **CLOs to be covered: CLO 1** |
| **Lab Title: While Loops** | **Teacher Name:** Hafiz Muhammad Mubashar |

## Lab Evaluation:

| **CLO1** | Apply appropriate programming techniques to create executable programs to solve well defined problems | | | | | |
|---|---|---|---|---|---|---|
| | **Level1** | **Level2** | **Level3** | **Level4** | **Level5** | **Level6** |
| (5) | | | | | | |
| | | | | | **Total** | /5 |

## Rubrics for Current Lab Evaluation

| Scale | Marks | Level | Rubric |
|---|---|---|---|
| Excellent | 5 | L1 | Submitted all lab tasks, BONUS task, have good understanding. |
| Very Good | 4 | L2 | Submitted the lab tasks but have good understanding |

| | | | |
|---|---|---|---|
| Good | 3 | L3 | Submitted the lab tasks but have weak understanding. |
| Basic | 2 | L4 | Submitted the lab tasks but have no understanding. |
| Barely Acceptable | 1 | L5 | Submitted only one lab task. |
| Not Acceptable | 0 | L6 | Did not attempt |

## LAB NO 3

### Lab Goals/Objectives:

- Learn the concept of the `while` loop in Python.
- Understand how to repeat statements using a condition.
- Apply `while` loops to solve simple programming problems.

### Equipment Required:

Computer system with PyCharm IDE and Python package installed on it.

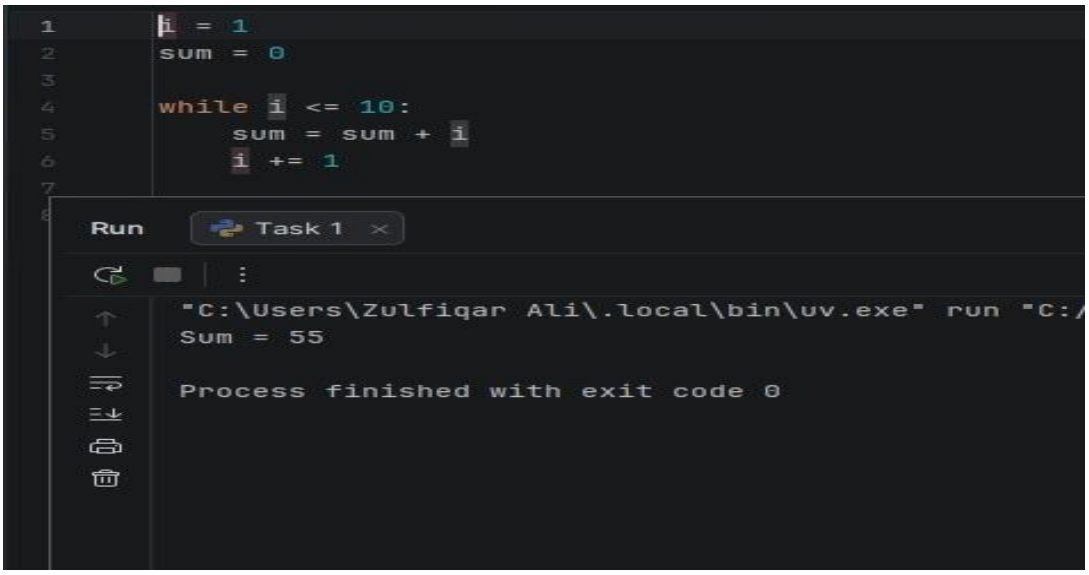
### Theory:

A `while` loop is used to execute a block of code repeatedly as long as a given condition is `True`. It checks the condition before each iteration. The loop stops when the condition becomes `False`.

### Lab Work:

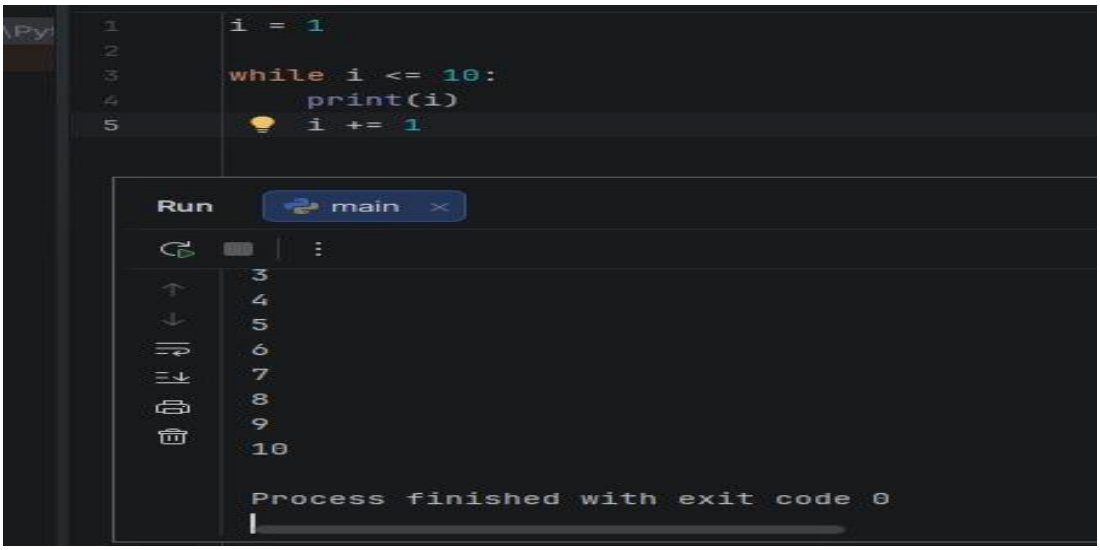
**Task 1:** Write a Python program to print numbers from 1 to 10 using a `while` loop

**Result:**



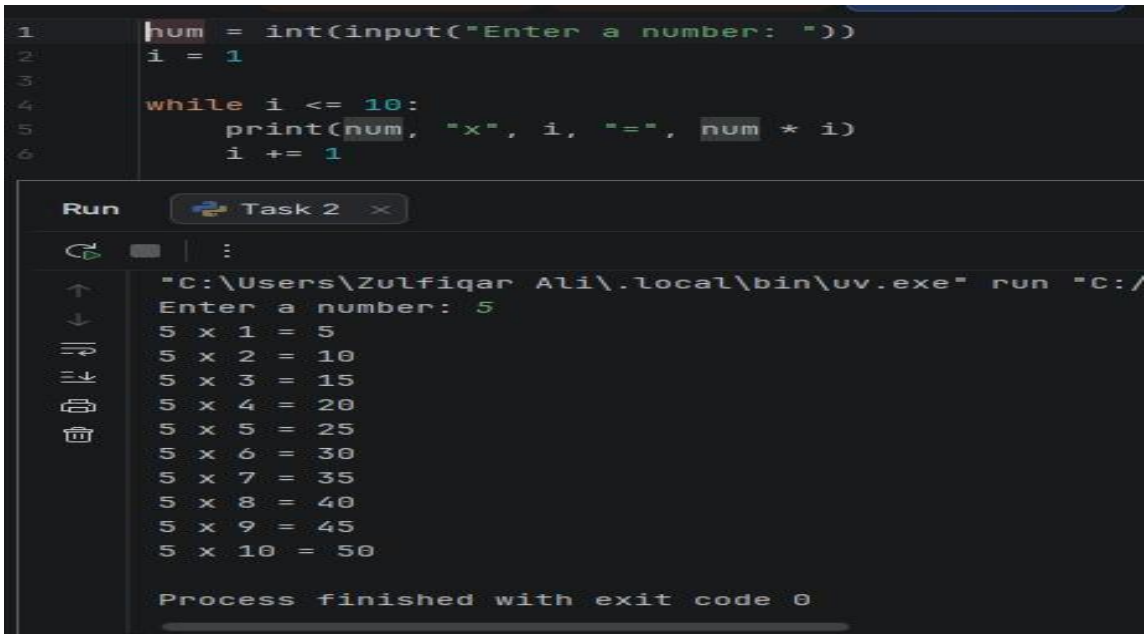

**Task 2:** Write a Python program to find the sum of the first 10 natural numbers using a `while` loop.

**Result:**



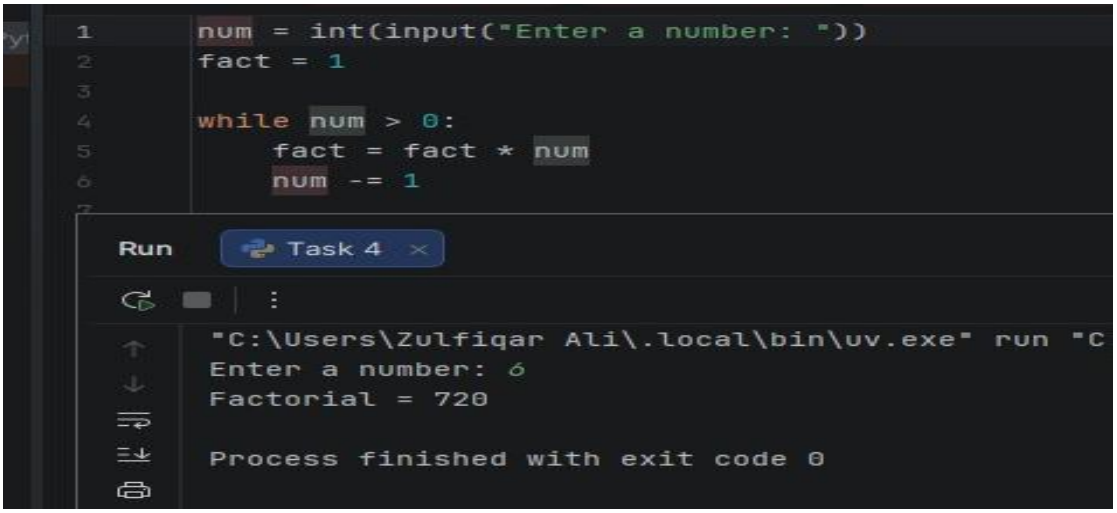

**Task 3:** Write a Python program to display the multiplication table of a number using a `while` loop.

**Result:**




**Task 4:** Write a Python program to calculate the factorial of a given number using a `while` loop.

**Result:**



## Conclusions:

The `while` loop is useful for repeating a set of statements as long as a specified condition is true. It helps solve problems where the number of iterations is not known beforehand. By performing this lab, students learned how to use `while` loops to implement repetition in Python programs.

| University Of Engineering and Technology, Lahore<br>Computer Engineering Department |
|---|

| | |
|---|---|
| **Course Name: Programing Fundamentals** | **Course Code:** CMPE-112L |
| **Assignment Type:** | **Dated: 23 March,2026** |
| **Semester: 1st** | **Session: Spring 2026** |
| **Lab/Project/Assignment #: 4** | **CLOs to be covered: CLO 1** |
| **Lab Title: Nested Loops** | **Teacher Name:** Hafiz Muhammad Mubashar |

**Lab Evaluation:**

| **CLO1** | Apply appropriate programming techniques to create executable programs to solve well defined problems | | | | | |
|---|---|---|---|---|---|---|
| | **Level1** | **Level2** | **Level3** | **Level4** | **Level5** | **Level6** |
| (5) | | | | | | |
| | | | | | **Total** | /5 |

**Rubrics for Current Lab Evaluation**

| Scale | Marks | Level | Rubric |
|---|---|---|---|
| Excellent | 5 | L1 | Submitted all lab tasks, BONUS task, have good understanding. |
| Very Good | 4 | L2 | Submitted the lab tasks but have good understanding |

| | | | |
|---|---|---|---|
| Good | 3 | L3 | Submitted the lab tasks but have weak understanding. |
| Basic | 2 | L4 | Submitted the lab tasks but have no understanding. |
| Barely Acceptable | 1 | L5 | Submitted only one lab task. |
| Not Acceptable | 0 | L6 | Did not attempt |

# LAB NO 4

## Lab Goals/Objectives:

By reading this manual, students will be able to:

- Understand the concept and working of nested loops in Python.
- Use nested `for` and `while` loops to solve programming problems.
- Create patterns, tables, and number sequences using nested loops.

## Equipment Required:

Computer system with PyCharm IDE and Python installed on it.

## Theory:

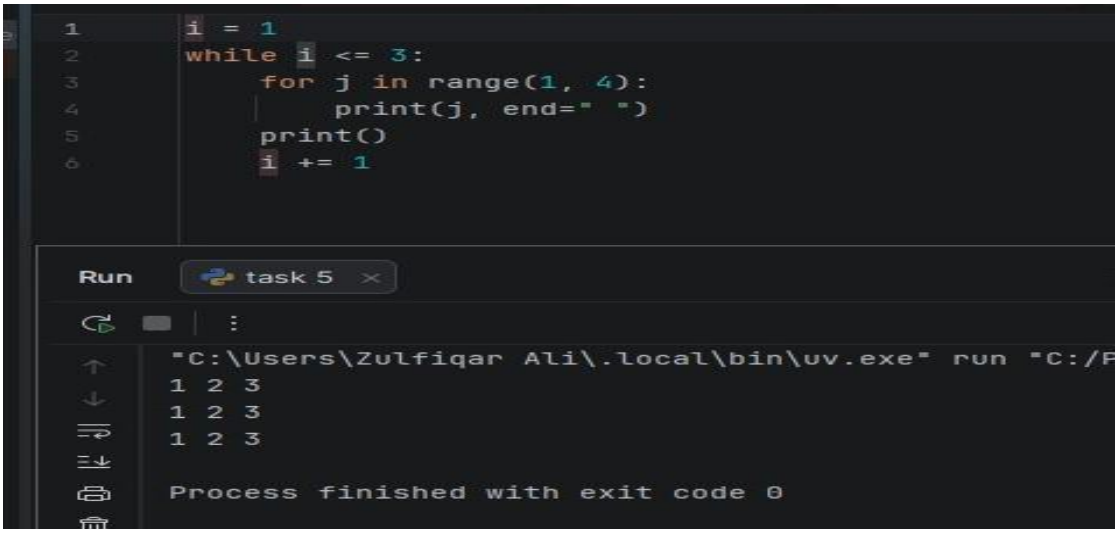
A nested loop is a loop inside another loop. The outer loop controls the number of times the inner loop executes. Nested loops are useful for working with patterns, tables, matrices, and repetitive tasks. Python allows nesting of both `for` and `while` loops.

## Lab Work:

### Task 1:

Write a Python program to print a square pattern of stars using nested `for` loops.

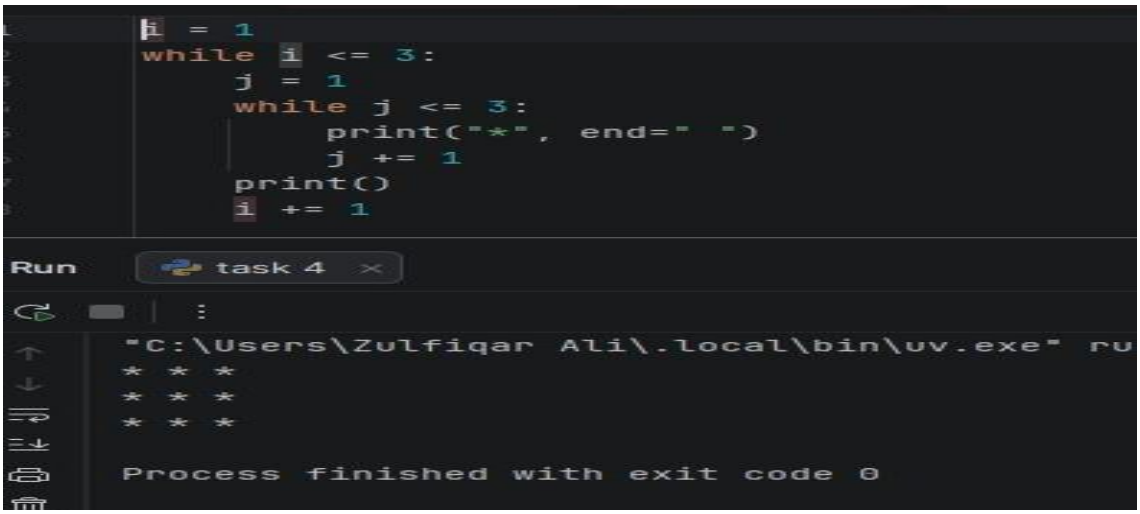
**Result:**




**Task 2:**

Write a Python program to display multiplication tables from 1 to 3 using nested `for` loops.

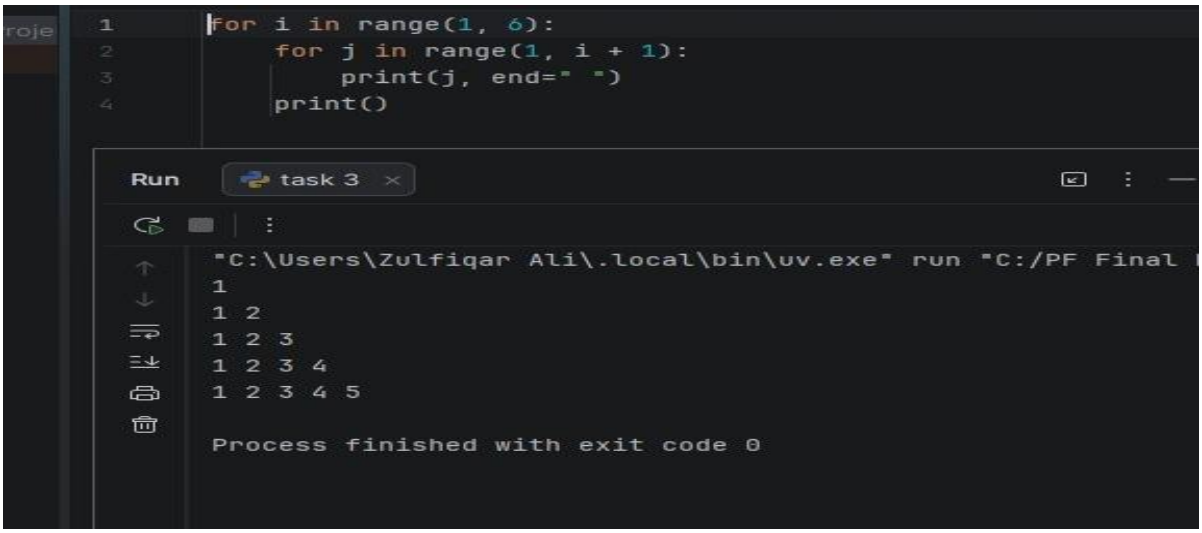
**Result:**




**Task 3:**

Write a Python program to print a triangle of numbers using nested `for` loops.

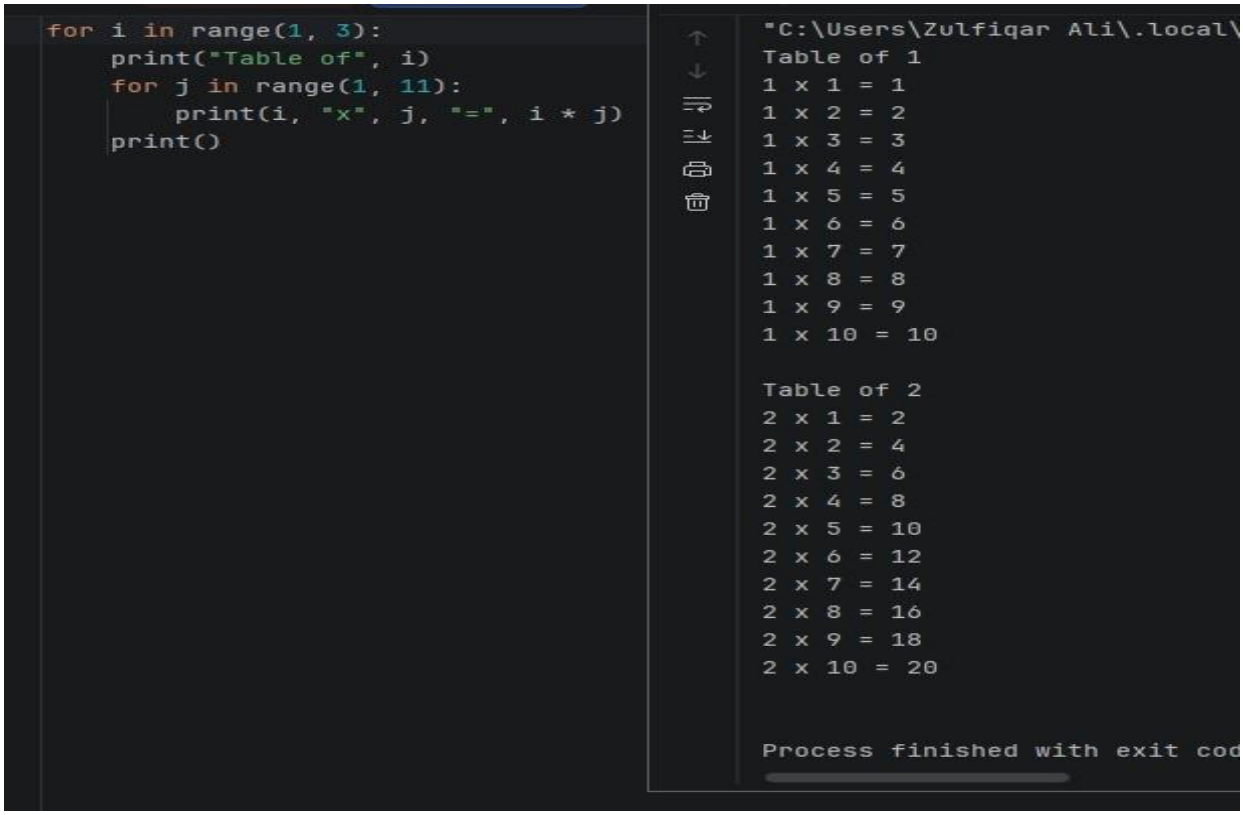
**Result:**




**Task 4:**

Write a Python program to print a 3×3 grid of stars using nested `while` loops.

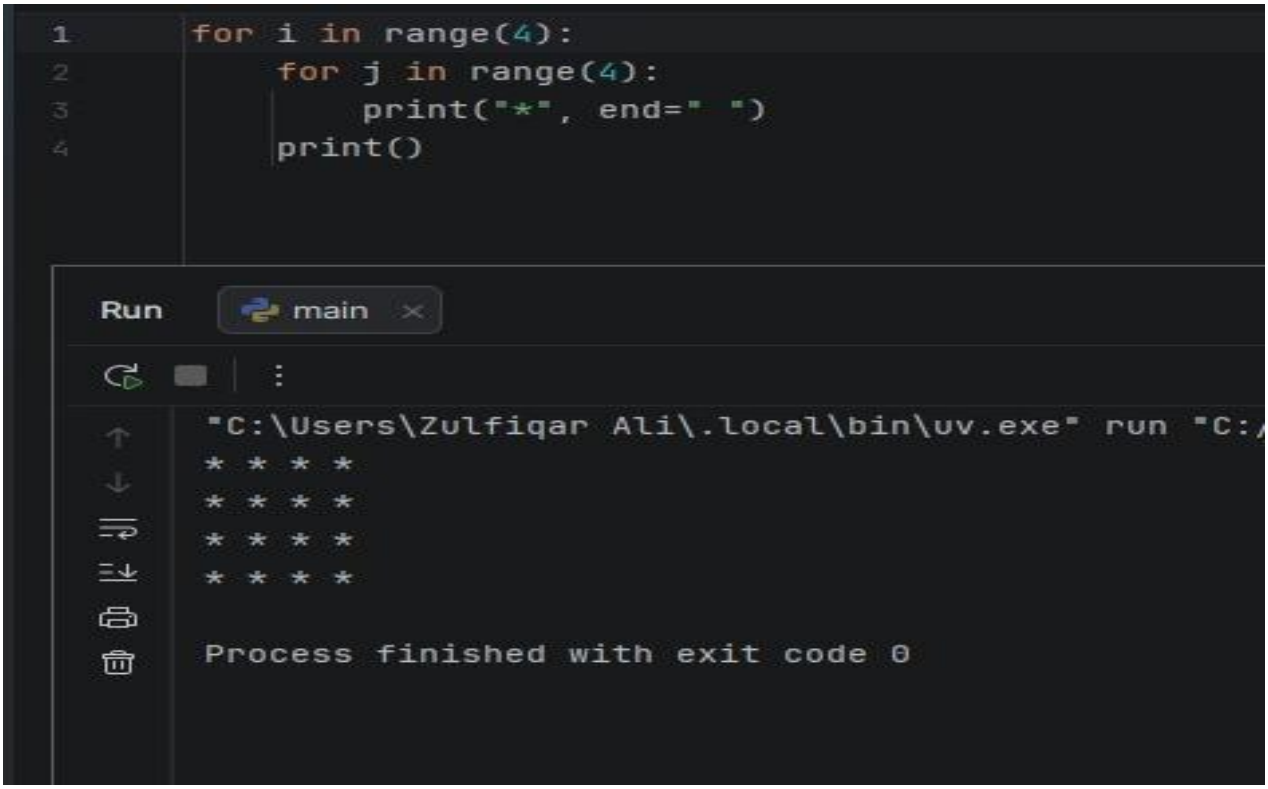
**Result:**



**Task 5:**

Write a Python program to print numbers using a `for` loop inside a `while` loop

**Result:**




## Conclusions:

Nested loops are loops placed inside other loops. They can be created using `for` loops, `while` loops, or a combination of both. Nested loops are commonly used for pattern printing, multiplication tables, matrices, and other repetitive programming tasks.

| University Of Engineering and Technology, Lahore<br>Computer Engineering Department |
|---|

| | |
|---|---|
| **Course Name: Programing Fundamentals** | **Course Code:** CMPE-112L |
| **Assignment Type:** | **Dated: 23 March,2026** |
| **Semester: 1st** | **Session: Spring 2026** |
| **Lab/Project/Assignment #: 5** | **CLOs to be covered: CLO 2** |
| **Lab Title: Functions** | **Teacher Name:** Hafiz Muhammad Mubashar |

**Lab Evaluation:**

| **CLO1** | Apply appropriate programming techniques to create executable programs to solve well defined problems | | | | | |
|---|---|---|---|---|---|---|
| | **Level1** | **Level2** | **Level3** | **Level4** | **Level5** | **Level6** |
| (5) | | | | | | |
| | | | | | **Total** | /5 |

**Rubrics for Current Lab Evaluation**

| Scale | Marks | Level | Rubric |
|---|---|---|---|
| Excellent | 5 | L1 | Submitted all lab tasks, BONUS task, have good understanding. |
| Very Good | 4 | L2 | Submitted the lab tasks but have good understanding |

| | | | |
|---|---|---|---|
| Good | 3 | L3 | Submitted the lab tasks but have weak understanding. |
| Basic | 2 | L4 | Submitted the lab tasks but have no understanding. |
| Barely Acceptable | 1 | L5 | Submitted only one lab task. |
| Not Acceptable | 0 | L6 | Did not attempt |

# LAB NO 5

## Lab Goals/Objectives:

By reading this manual, students will be able to:

- Understand the concept and purpose of functions in Python.
- Create and use user-defined functions with parameters and return values.
- Apply functions to organize and simplify Python programs.

## Equipment Required:

Computer system with PyCharm IDE and Python installed on it.

## Theory:

A function is a block of code that performs a specific task. Functions help make programs more organized, reusable, and easier to maintain. In Python, functions are defined using the `def` keyword. A function can accept inputs called parameters and can return a value using the `return` statement. Functions reduce code repetition and improve program readability.

## Lab Work:

### Task 1:

Write a Python program to create and call a function that displays a welcome message.

**Result:**

```
def maximum(a :{__gt__} , b):
    if a > b:
        return a
    else:
        return b

print("Maximum =", maximum(15, 10))
```

```
Run  Task 6
"C:\Users\Zulfiqar Ali\.local\bin\uv.exe" run "C:/PF Final Pr
Maximum = 15

Process finished with exit code 0
```

**Task 2:**

Write a Python program to create a function that takes two numbers as parameters and displays their sum.

**Result:**

```
def area(length :{__mul__} , width):
    return length * width

l = float(input("Enter length: "))
w = float(input("Enter width: "))

print("Area =", area(l, w))
```

```
Run  Task 5
"C:\Users\Zulfiqar Ali\.local\bin\uv.exe" run "C:/PF Final Pr
Enter length: 10
Enter width: 5
Area = 50.0

Process finished with exit code 0
```

**Task 3:**

Write a Python program to create a function that returns the square of a number.

**Result:**

```python
def check_number(num :{__mod__} ):
    if num % 2 == 0:
        print("Even Number")
    else:
        print("Odd Number")

check_number(8)
```

```
Run  Task 4
"C:\Users\Zulfiqar Ali\.local\bin\uv.exe" run "C:/PF
Even Number

Process finished with exit code 0
```

**Task 4:**

Write a Python program to create a function that checks whether a number is even or odd.

**Result:**

```python
def square(num :{__mul__} ):
    return num * num

result = square(5)
print("Square =", result)
```

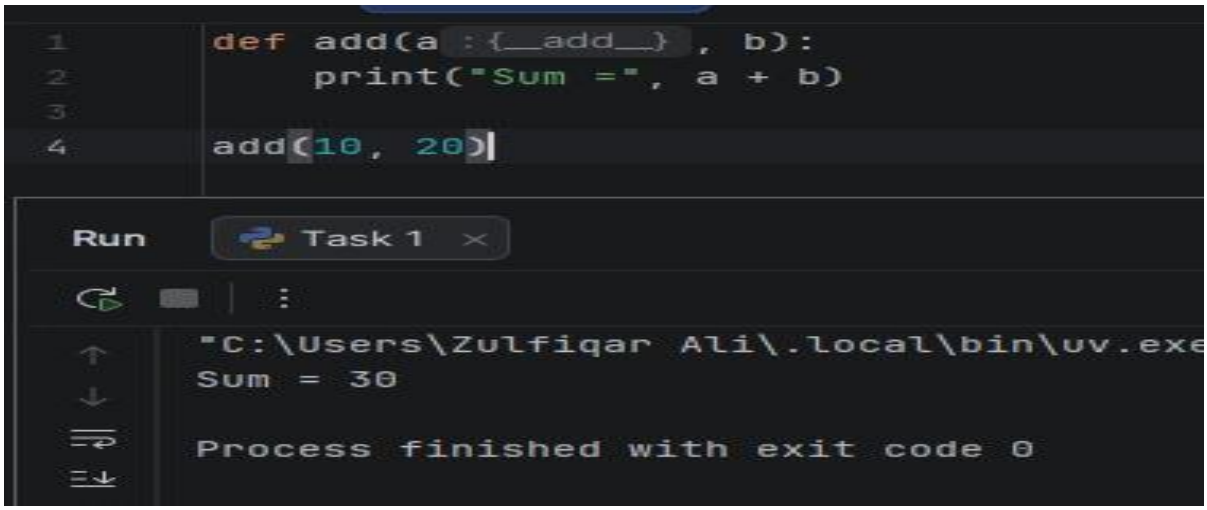
```
Run  Task 2
"C:\Users\Zulfiqar Ali\.local\bin\uv.exe" run "C:/P
Square = 25

Process finished with exit code 0
```

**Task 5:**

Write a Python program to create a function that calculates the area of a rectangle.

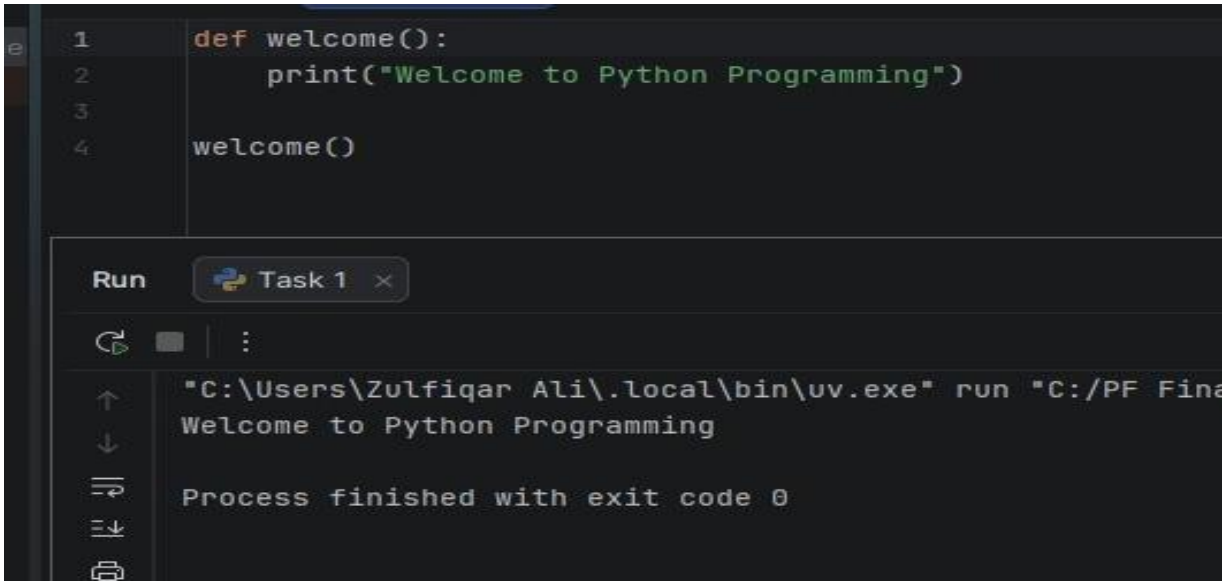
**Result:**




**Task 6:**

Write a Python program to create a function that returns the larger of two numbers.

**Result:**



## Conclusions:

Functions are an important feature of Python that allow programmers to divide a program into smaller, reusable blocks of code. They improve code organization, reduce repetition, and make programs easier to understand and maintain. By using parameters and return values, functions can perform a wide variety of tasks efficiently.

| University Of Engineering and Technology, Lahore<br>Computer Engineering Department |
|---|

| | |
|---|---|
| **Course Name: Programing Fundamental** | **Course Code:** CMPE-112L |
| **Assignment Type:** | **Dated: 23 March,2026** |
| **Semester: 1st** | **Session: Spring 2026** |
| **Lab/Project/Assignment #: 6** | **CLOs to be covered: CLO 2** |
| **Lab Title: Local and Global Variables** | **Teacher Name:** Hafiz Muhammad Mubashar |

## Lab Evaluation:

| **CLO1** | Apply appropriate programming techniques to create executable programs to solve well defined problems | | | | | |
|---|---|---|---|---|---|---|
| | **Level1** | **Level2** | **Level3** | **Level4** | **Level5** | **Level6** |
| (5) | | | | | | |
| | | | | | **Total** | /5 |

## Rubrics for Current Lab Evaluation

| Scale | Marks | Level | Rubric |
|---|---|---|---|
| Excellent | 5 | L1 | Submitted all lab tasks, BONUS task, have good understanding. |
| Very Good | 4 | L2 | Submitted the lab tasks but have good understanding |

| | | | |
|---|---|---|---|
| Good | 3 | L3 | Submitted the lab tasks but have weak understanding. |
| Basic | 2 | L4 | Submitted the lab tasks but have no understanding. |
| Barely Acceptable | 1 | L5 | Submitted only one lab task. |
| Not Acceptable | 0 | L6 | Did not attempt |

# LAB NO 6

## Lab Goals/Objectives:

By reading this manual, students will be able to:

- Understand the difference between local and global variables in Python.
- Use local and global variables correctly within functions.
- Apply variable scope concepts in Python programs.

## Equipment Required:

Computer system with PyCharm IDE and Python installed on it.

## Theory:

Variables in Python can have different scopes depending on where they are declared.

A **local variable** is declared inside a function and can only be accessed within that function. It exists only while the function is running.

A **global variable** is declared outside all functions and can be accessed from anywhere in the program. To modify a global variable inside a function, the `global` keyword is used.

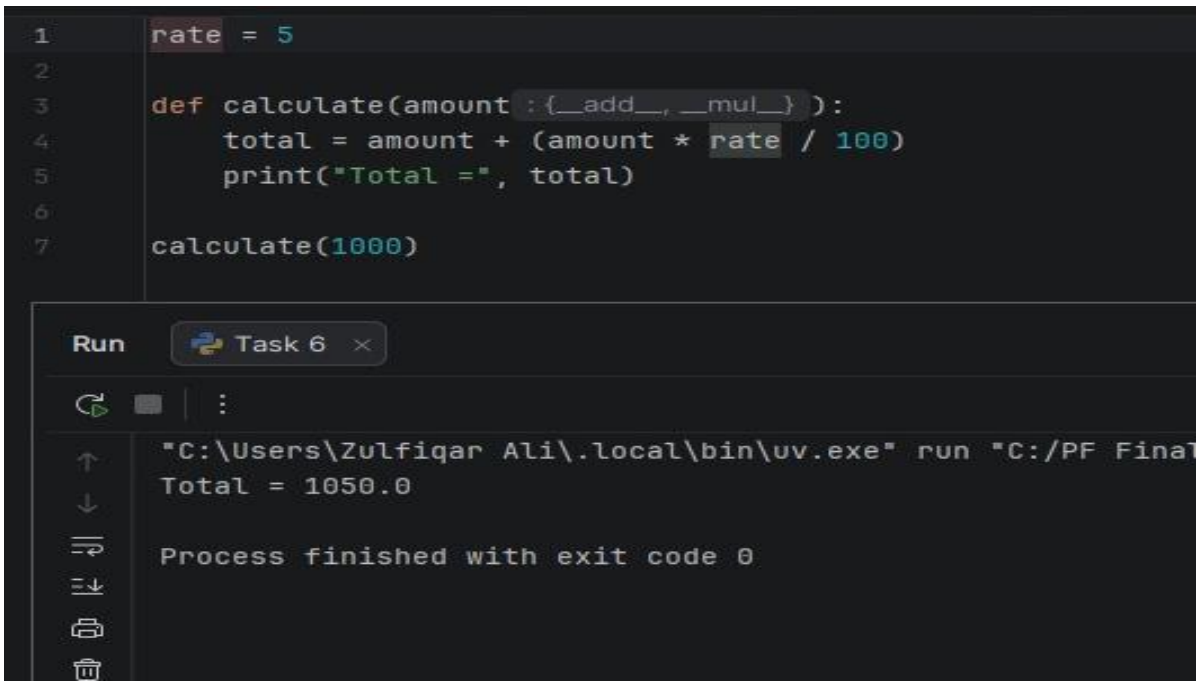
Understanding local and global variables helps programmers manage data effectively and avoid scope-related errors.

## Lab Work:

**Task 1:**

Write a Python program to demonstrate a local variable inside a function.

**Result:**




**Task 2:**

Write a Python program to demonstrate a global variable.

**Result:**

```
def first():
    num = 10
    print("First Function:", num)

def second():
    num = 20
    print("Second Function:", num)

first()
second()
```

```
Run  Task 5
"C:\Users\Zulfiqar Ali\.local\bin\uv.exe" run "C:/PF Fina
First Function: 10
Second Function: 20

Process finished with exit code 0
```

**Task 3:**

Write a Python program to show the difference between local and global variables having the same name.

**Result:**

```
count = 0

def increase():
    global count
    count = count + 1

increase()
print("Count =", count)
```

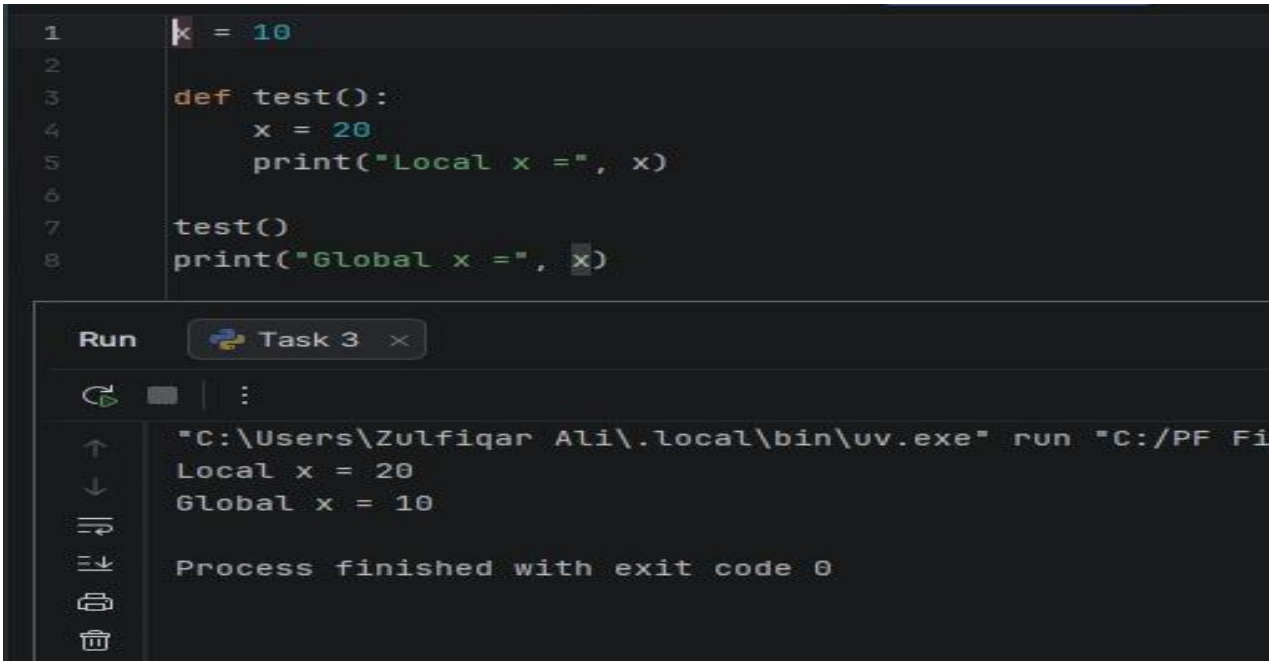
```
Run  Task 4
"C:\Users\Zulfiqar Ali\.local\bin\uv.exe" run "C:/PF Fin
Count = 1

Process finished with exit code 0
```

**Task 4:**

Write a Python program to modify a global variable using the `global` keyword.

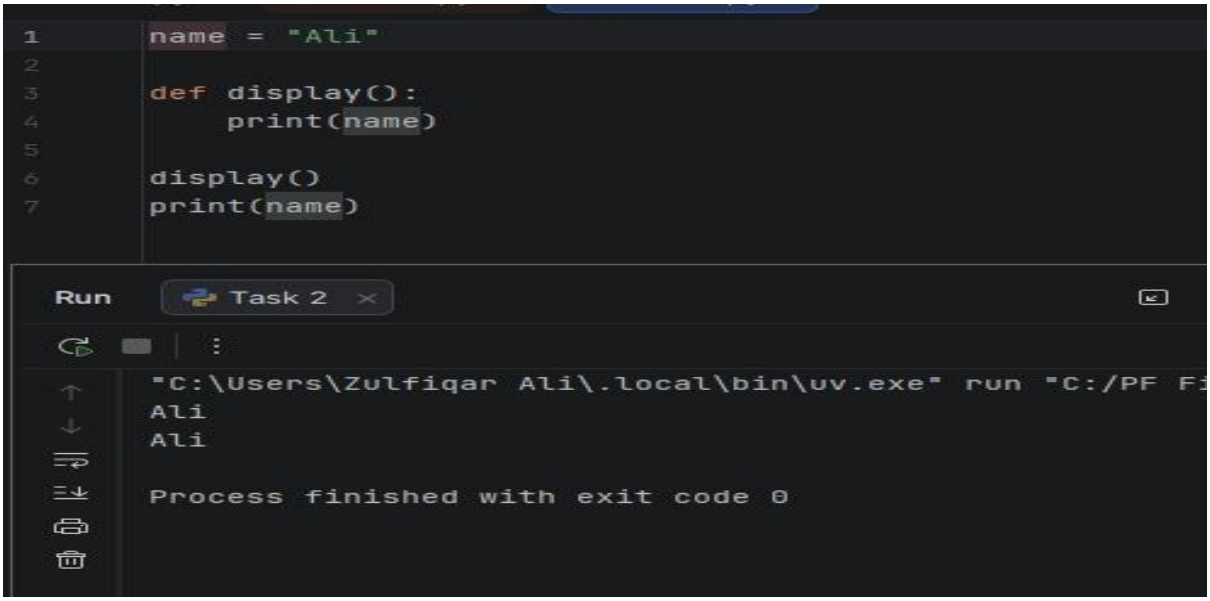
**Result:**




**Task 5:**

Write a Python program to demonstrate that local variables are independent in different functions.

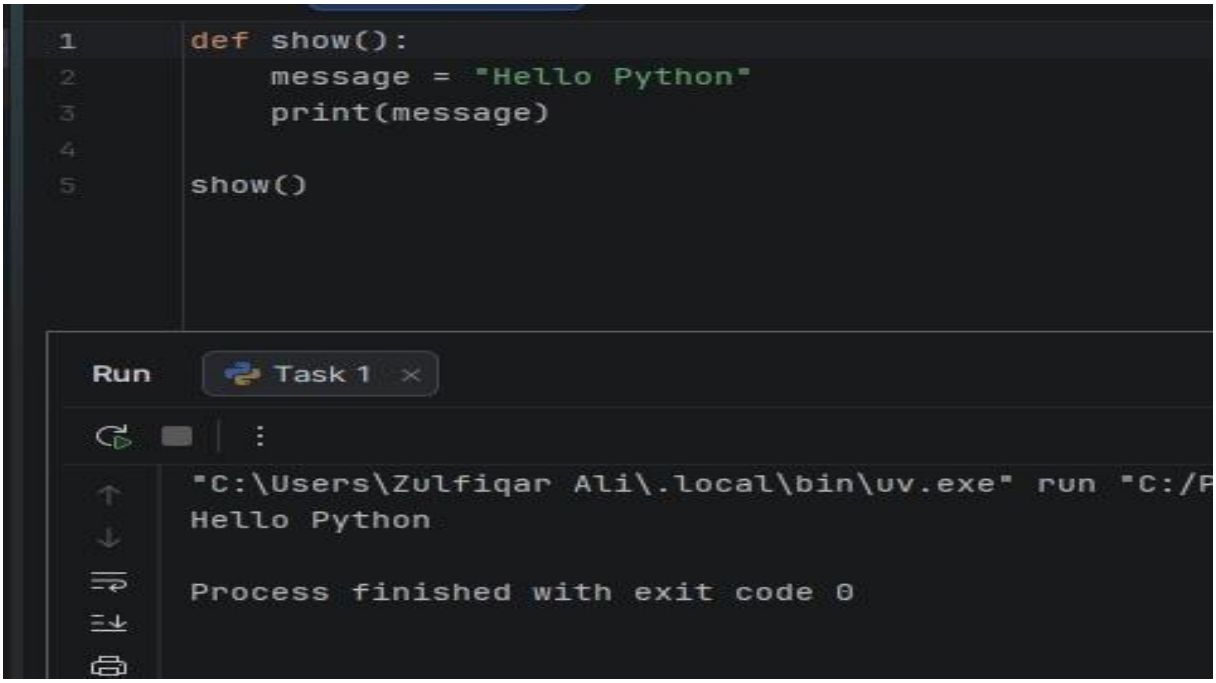
**Result:**




**Task 6:**

Write a Python program that uses a global variable in a calculation function.

**Result:**




**Conclusions:**

Local variables are created inside functions and can only be used within those functions, while global variables are declared outside functions and can be accessed throughout the program. Understanding the scope of variables helps in writing efficient, organized, and error-free Python programs.

| University Of Engineering and Technology, Lahore<br>Computer Engineering Department |
|---|

| | |
|---|---|
| **Course Name: Programing Fundamentals** | **Course Code:** CMPE-112L |
| **Assignment Type:** | **Dated: 23 March,2026** |
| **Semester: 1st** | **Session: Spring 2026** |
| **Lab/Project/Assignment #: 7** | **CLOs to be covered: CLO 2** |
| **Lab Title: Lists, Tuples** | **Teacher Name:** Hafiz Muhammad Mubashar |

**Lab Evaluation:**

| **CLO1** | Apply appropriate programming techniques to create executable programs to solve well defined problems | | | | | |
|---|---|---|---|---|---|---|
| | **Level1** | **Level2** | **Level3** | **Level4** | **Level5** | **Level6** |
| (5) | | | | | | |
| | | | | | **Total** | /5 |

**Rubrics for Current Lab Evaluation**

| Scale | Marks | Level | Rubric |
|---|---|---|---|
| Excellent | 5 | L1 | Submitted all lab tasks, BONUS task, have good understanding. |
| Very Good | 4 | L2 | Submitted the lab tasks but have good understanding |

| | | | |
|---|---|---|---|
| Good | 3 | L3 | Submitted the lab tasks but have weak understanding. |
| Basic | 2 | L4 | Submitted the lab tasks but have no understanding. |
| Barely Acceptable | 1 | L5 | Submitted only one lab task. |
| Not Acceptable | 0 | L6 | Did not attempt |

# LAB NO 7

## Lab Goals/Objectives:

By reading this manual, students will be able to:

- Understand the concepts of Lists and Tuples in Python.
- Perform basic operations on Lists and Tuples.
- Differentiate between mutable Lists and immutable Tuples.

## Equipment Required:

Computer system with PyCharm IDE and Python installed on it.

## Theory:

Lists and Tuples are Python data structures used to store multiple items in a single variable.

A **List** is an ordered and mutable collection, meaning its elements can be changed, added, or removed after creation. Lists are created using square brackets `[]`.

A **Tuple** is an ordered and immutable collection, meaning its elements cannot be modified after creation. Tuples are created using parentheses `()`.

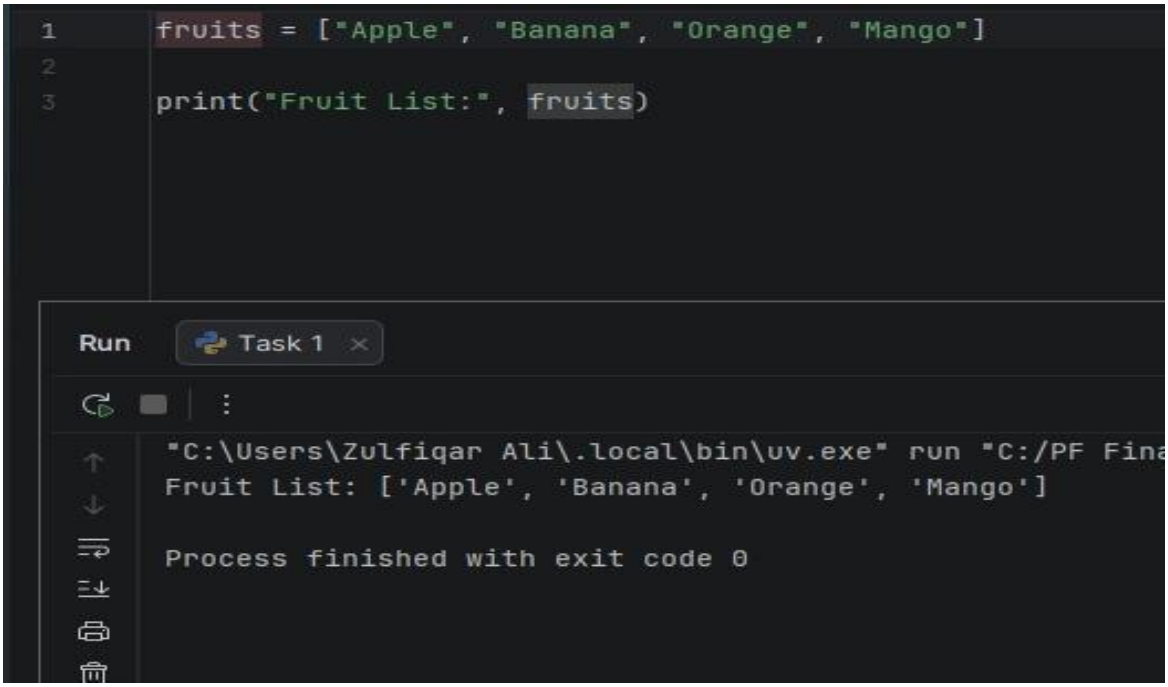
Lists are useful when data needs to be modified, while Tuples are preferred when data should remain constant.

## Lab Work:

**Task 1:**

Write a Python program to create and display a list of fruits.

**Result:**




**Task 2:**

Write a Python program to add and remove elements from a list.

**Result:**

```python
numbers = [10, 20, 30]

numbers.append(40)
numbers.remove(20)

print("Updated List:", numbers)
```

```
Run  Task 2
"C:\Users\Zulfiqar Ali\.local\bin\uv.exe" run "C:/PF
Updated List: [10, 30, 40]

Process finished with exit code 0
```

**Task 3:**

Write a Python program to display all elements of a list using a loop.

**Result:**

```python
colors = ["Red", "Green", "Blue"]

for color in colors:
    print(color)
```

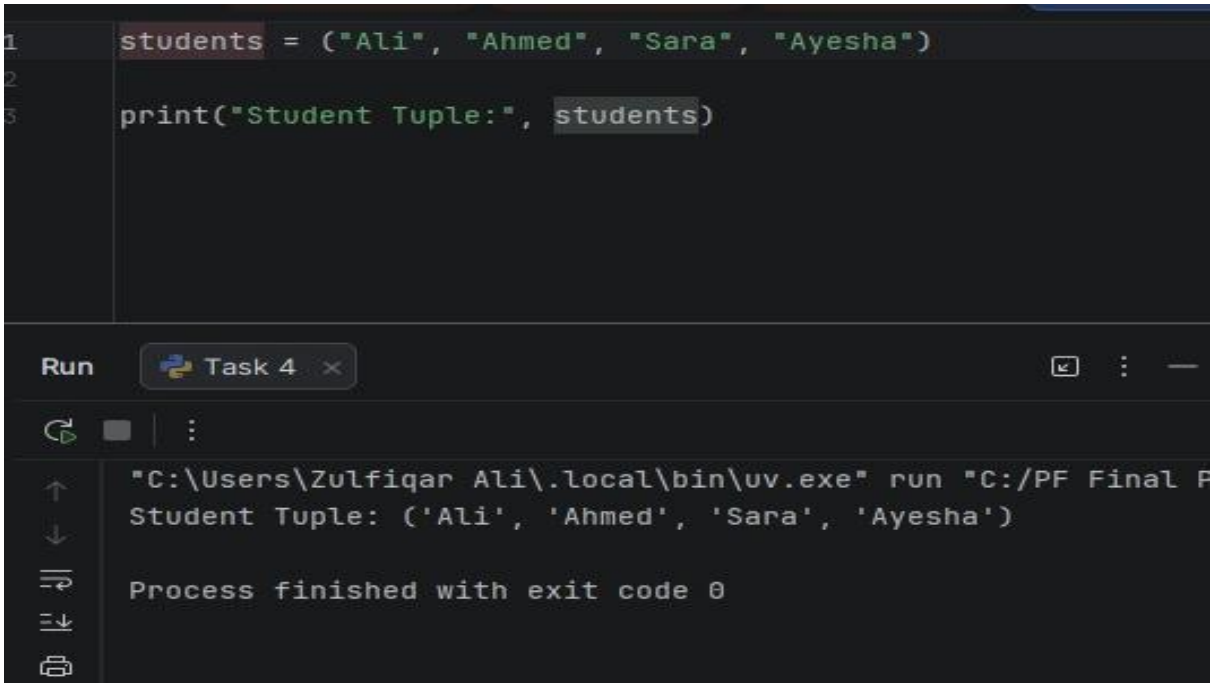
```
Run  Task 3
"C:\Users\Zulfiqar Ali\.local\bin\uv.exe" run
Red
Green
Blue

Process finished with exit code 0
```

**Task 4:**

Write a Python program to create and display a tuple of student names.

**Result:**




**Task 5:**

Write a Python program to access elements of a tuple using indexing.

**Result:**

```
numbers = (100, 200, 300, 400)

print("First Element:", numbers[0])
print("Last Element:", numbers[-1])
```

```
Run  Task 5
"C:\Users\Zulfiqar Ali\.local\bin\uv.exe" run "C
First Element: 100
Last Element: 400

Process finished with exit code 0
```

## Task 6:

Write a Python program to demonstrate the difference between a List and a Tuple.

**Result:**

```
my_list = [1, 2, 3]
my_tuple = (1, 2, 3)

my_list[0] = 10

print("List:", my_list)
print("Tuple:", my_tuple)
```

```
Run  Task 6
"C:\Users\Zulfiqar Ali\.local\bin\uv.exe" run "C
List: [10, 2, 3]
Tuple: (1, 2, 3)

Process finished with exit code 0
```

## Conclusions:

Lists and Tuples are important Python data structures used to store collections of data. Lists are mutable and allow modifications, while Tuples are immutable and provide data security. Understanding both helps programmers choose the appropriate structure based on application requirements.

| University Of Engineering and Technology, Lahore<br>Computer Engineering Department |
|---|

| | |
|---|---|
| **Course Name: Programing Fundamentals** | **Course Code:** CMPE-112L |
| **Assignment Type:** | **Dated: 23 March,2026** |
| **Semester: 1st** | **Session: Spring 2026** |
| **Lab/Project/Assignment #: 8** | **CLOs to be covered: CLO 3** |
| **Lab Title: Sets, Dictionaries** | **Teacher Name:** Hafiz Muhammad Mubashar |

## Lab Evaluation:

| **CLO1** | Apply appropriate programming techniques to create executable programs to solve well defined problems | | | | | |
|---|---|---|---|---|---|---|
| | **Level1** | **Level2** | **Level3** | **Level4** | **Level5** | **Level6** |
| (5) | | | | | | |
| | | | | | **Total** | /5 |

## Rubrics for Current Lab Evaluation

| Scale | Marks | Level | Rubric |
|---|---|---|---|
| Excellent | 5 | L1 | Submitted all lab tasks, BONUS task, have good understanding. |
| Very Good | 4 | L2 | Submitted the lab tasks but have good understanding |

| | | | |
|---|---|---|---|
| Good | 3 | L3 | Submitted the lab tasks but have weak understanding. |
| Basic | 2 | L4 | Submitted the lab tasks but have no understanding. |
| Barely Acceptable | 1 | L5 | Submitted only one lab task. |
| Not Acceptable | 0 | L6 | Did not attempt |

# LAB NO 8

## Lab Goals/Objectives:

By reading this manual, students will be able to:

- Understand the concepts of Sets and Dictionaries in Python.
- Perform basic operations on Sets and Dictionaries.
- Use Sets and Dictionaries to store and manage data efficiently.

## Equipment Required:

Computer system with PyCharm IDE and Python installed on it.

## Theory:

**Sets** and **Dictionaries** are built-in data structures in Python.

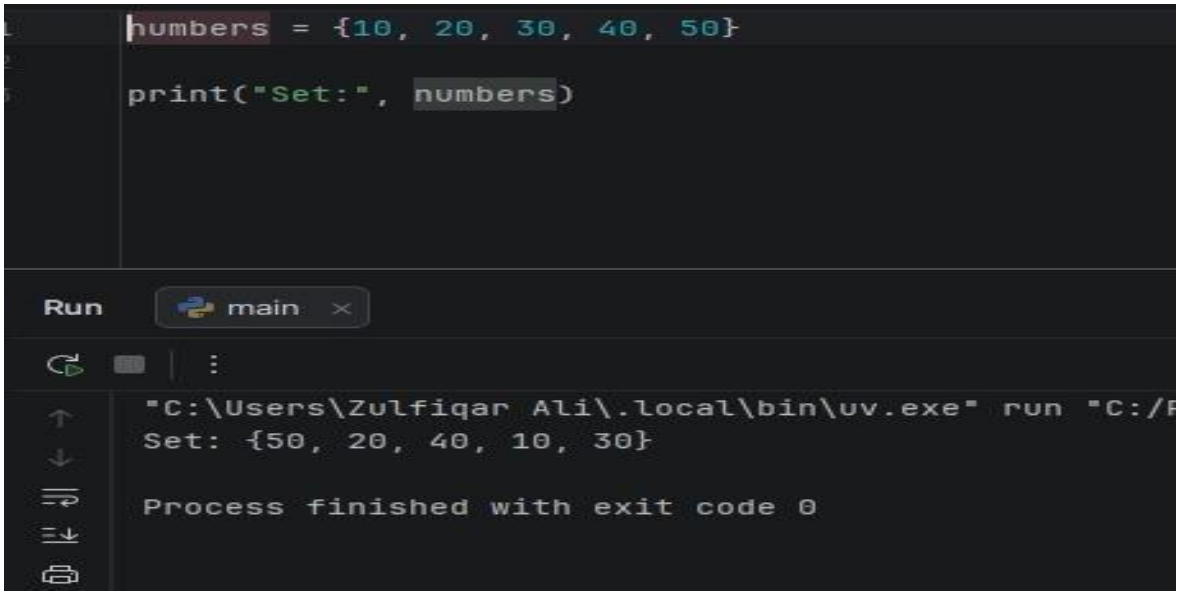
A **Set** is an unordered collection of unique elements. Sets do not allow duplicate values and are created using curly braces `{}` or the `set()` function. They are useful for storing distinct items and performing set operations.A **Dictionary** is a collection of key-value pairs. Each key is unique and is used to access its corresponding value. Dictionaries are created using curly braces `{}` with keys and values separated by colons `:`.Sets are commonly used for removing duplicates, while Dictionaries are used to store and retrieve data efficiently using keys.

## Lab Work:

### Task 1:

Write a Python program to create and display a set of numbers.

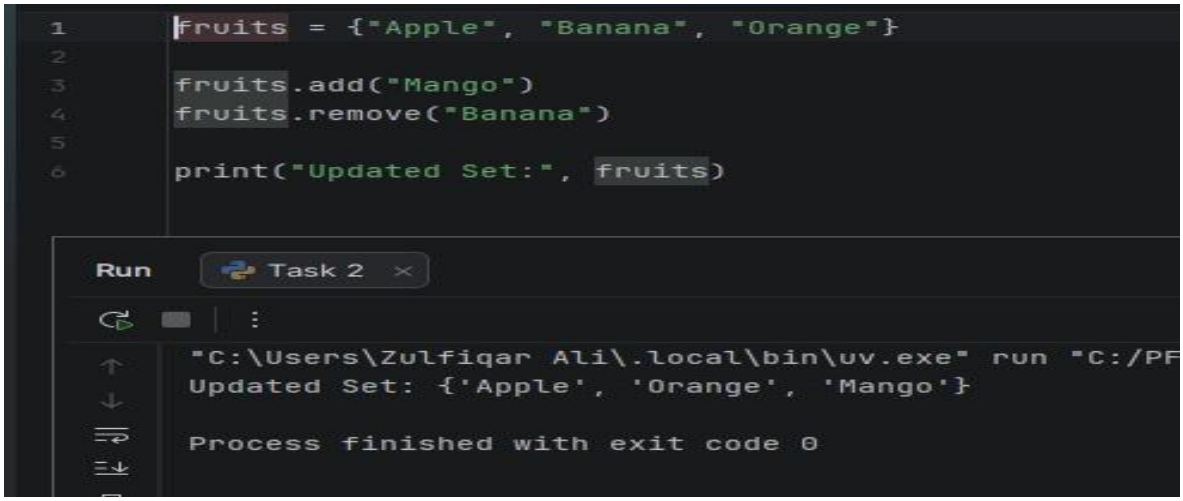
**Result:**




**Task 2:**

Write a Python program to add and remove elements from a set.

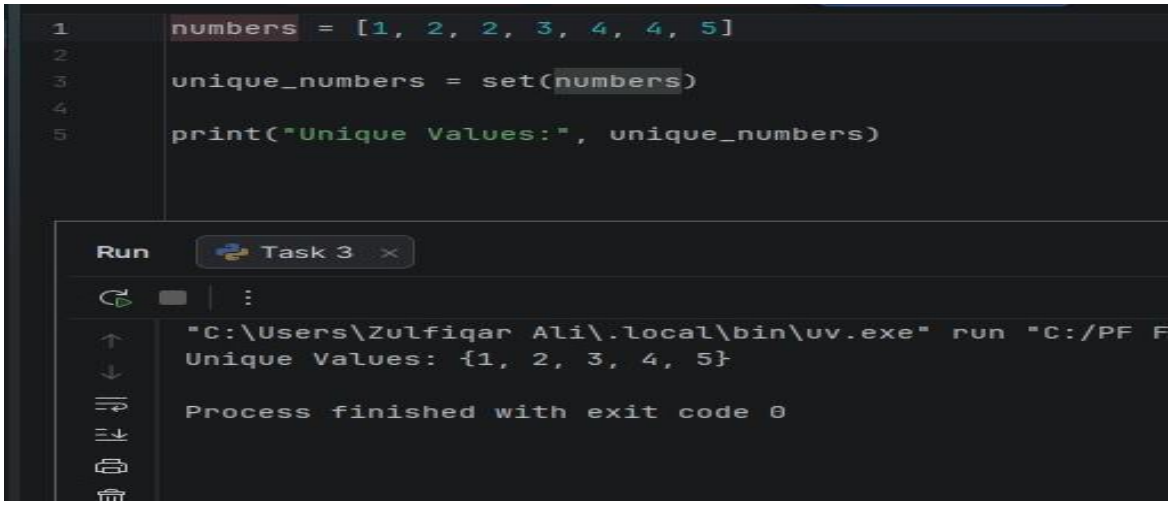
**Result:**




**Task 3:**

Write a Python program to remove duplicate values from a list using a set.

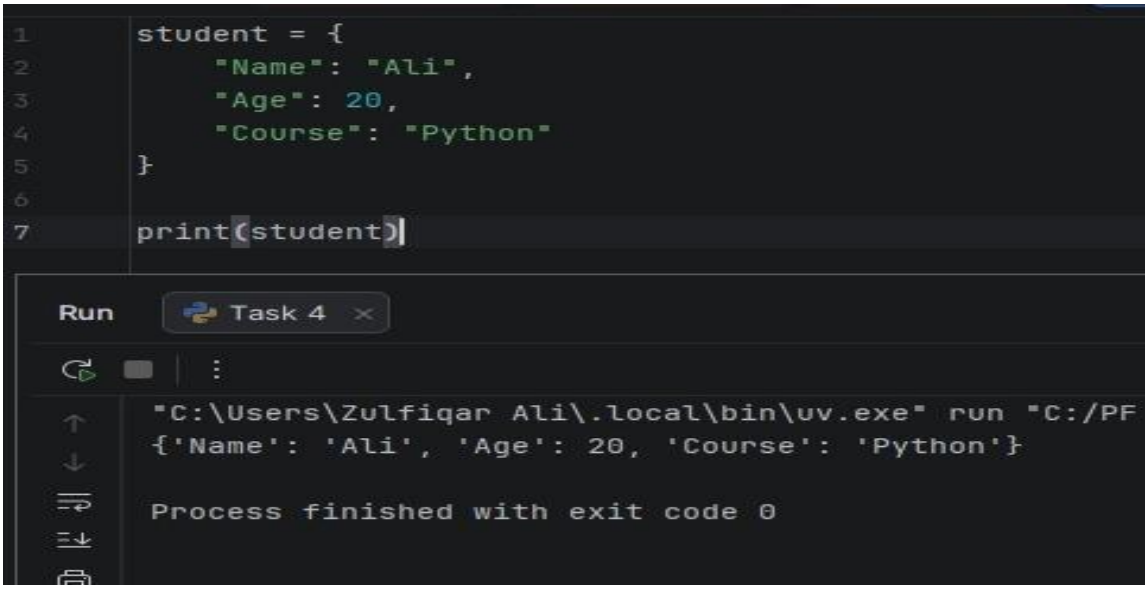
**Result:**




**Task 4:**

Write a Python program to create and display a dictionary of student information.

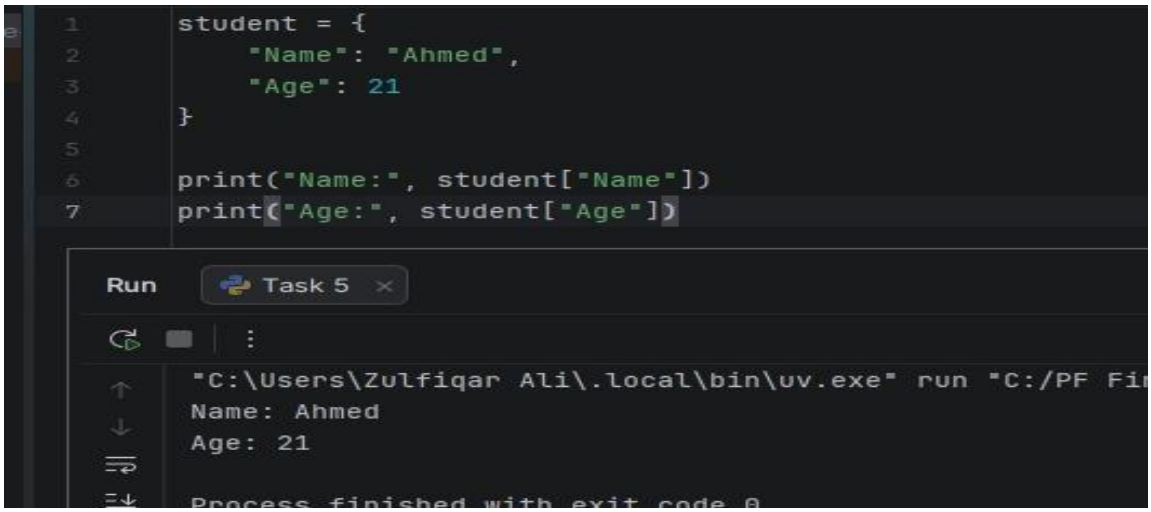
**Result:**




**Task 5:**

Write a Python program to access values from a dictionary using keys.

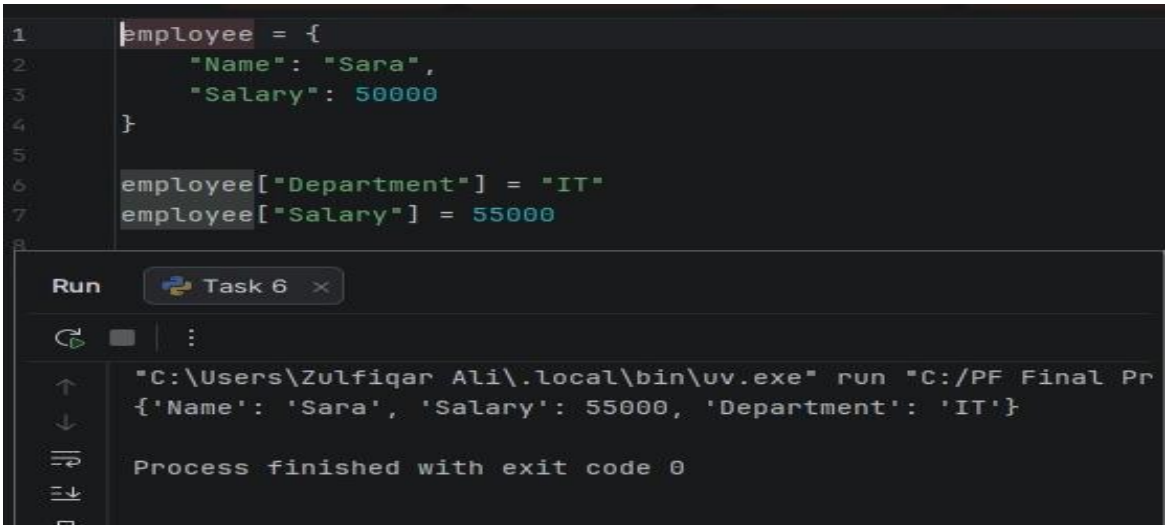
**Result:**




**Task 6:**

Write a Python program to add and update items in a dictionary.

**Result:**



## Conclusions:

Sets and Dictionaries are powerful Python data structures. Sets store unique values and help eliminate duplicates, while Dictionaries store data as key-value pairs for fast and efficient access. Understanding these structures helps in organizing and managing data effectively in Python programs.

| University Of Engineering and Technology, Lahore<br>Computer Engineering Department |
|---|

| | |
|---|---|
| **Course Name: Programing Fundamentals** | **Course Code:** CMPE-112L |
| **Assignment Type:** | **Dated: 23 March,2026** |
| **Semester: 1st** | **Session: Spring 2026** |
| **Lab/Project/Assignment #: 10** | **CLOs to be covered: CLO 2** |
| **Lab Title: Recursions** | **Teacher Name:** Hafiz Muhammad Mubashar |

**Lab Evaluation:**

| **CLO1** | Apply appropriate programming techniques to create executable programs to solve well defined problems | | | | | |
|---|---|---|---|---|---|---|
| | **Level1** | **Level2** | **Level3** | **Level4** | **Level5** | **Level6** |
| (5) | | | | | | |
| | | | | | **Total** | /5 |

**Rubrics for Current Lab Evaluation**

| Scale | Marks | Level | Rubric |
|---|---|---|---|
| Excellent | 5 | L1 | Submitted all lab tasks, BONUS task, have good understanding. |
| Very Good | 4 | L2 | Submitted the lab tasks but have good understanding |

| | | | |
|---|---|---|---|
| Good | 3 | L3 | Submitted the lab tasks but have weak understanding. |
| Basic | 2 | L4 | Submitted the lab tasks but have no understanding. |
| Barely Acceptable | 1 | L5 | Submitted only one lab task. |
| Not Acceptable | 0 | L6 | Did not attempt |

# LAB NO 10

## Lab Goals/Objectives:

By reading this manual, students will be able to:

- Understand the concept of recursion in Python.
- Develop recursive functions to solve different problems.
- Differentiate between recursive and iterative approaches.

## Equipment Required:

Computer system with PyCharm IDE and Python installed on it.

## Theory:

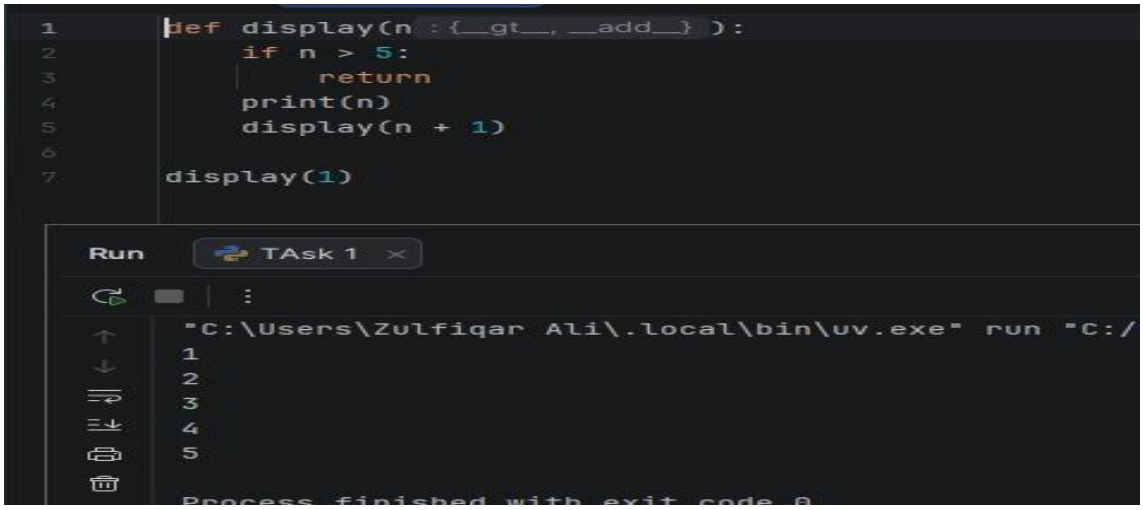
Recursion is a programming technique in which a function calls itself to solve a problem. A recursive function consists of two parts: the **base case**, which stops the recursion, and the **recursive case**, where the function calls itself with a smaller or simpler input. Recursion is commonly used for mathematical calculations, searching, sorting, and problem-solving tasks that can be divided into smaller subproblems.

## Lab Work:

### Task 1:

Write a Python program to display numbers from 1 to 5 using recursion.

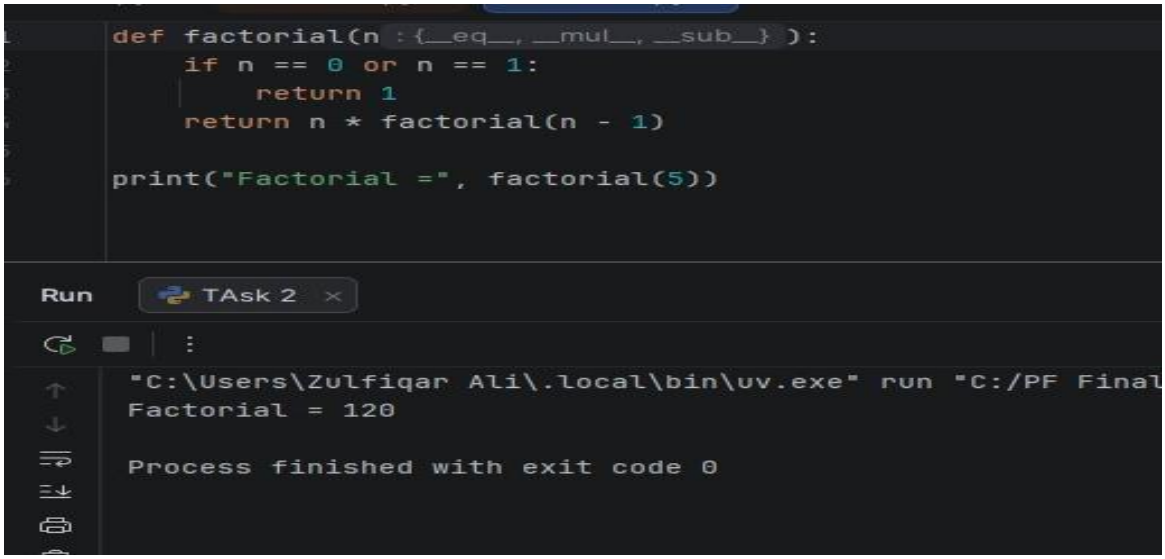
**Result:**




**Task 2:**

Write a Python program to find the factorial of a number using recursion.

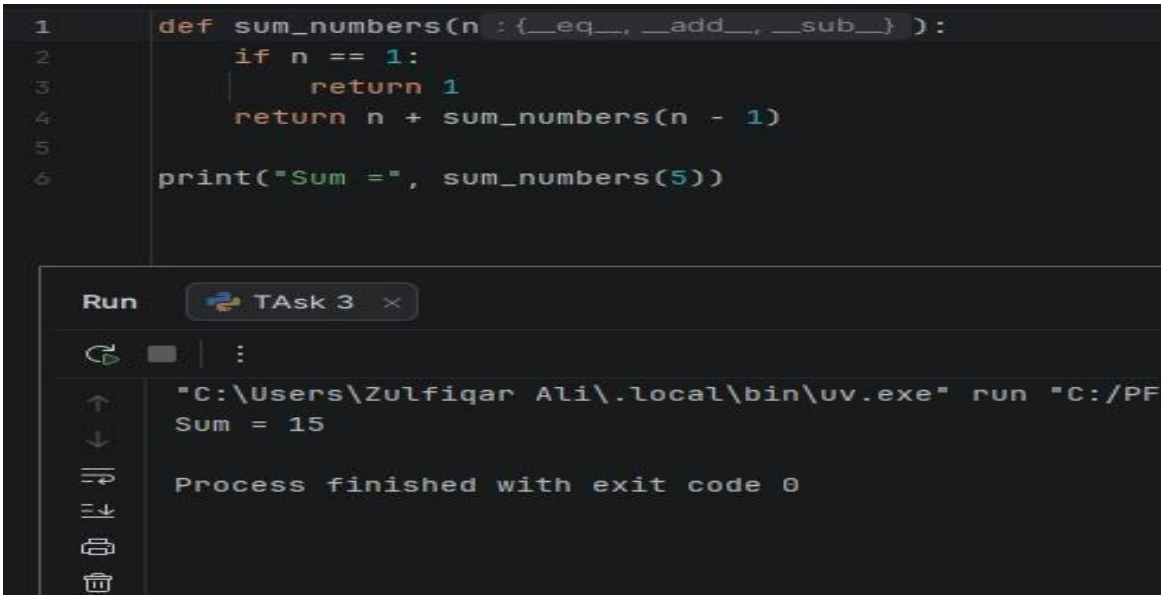
**Result:**




**Task 3:**

Write a Python program to find the sum of the first n natural numbers using recursion.

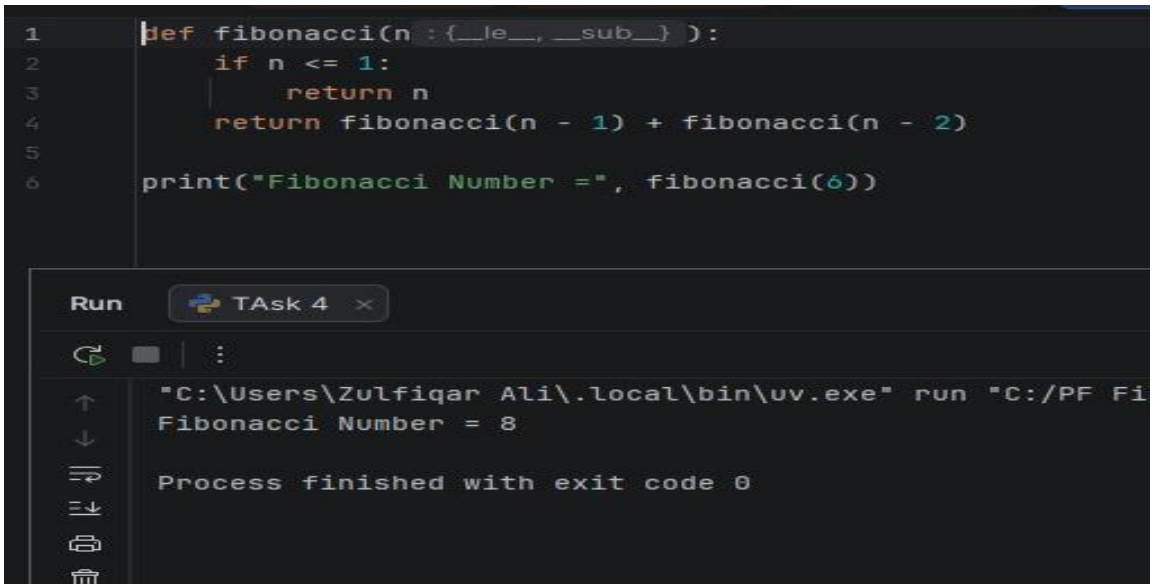
**Result:**




**Task 4:**

Write a Python program to find the nth Fibonacci number using recursion.

**Result:**




**Task 5:**

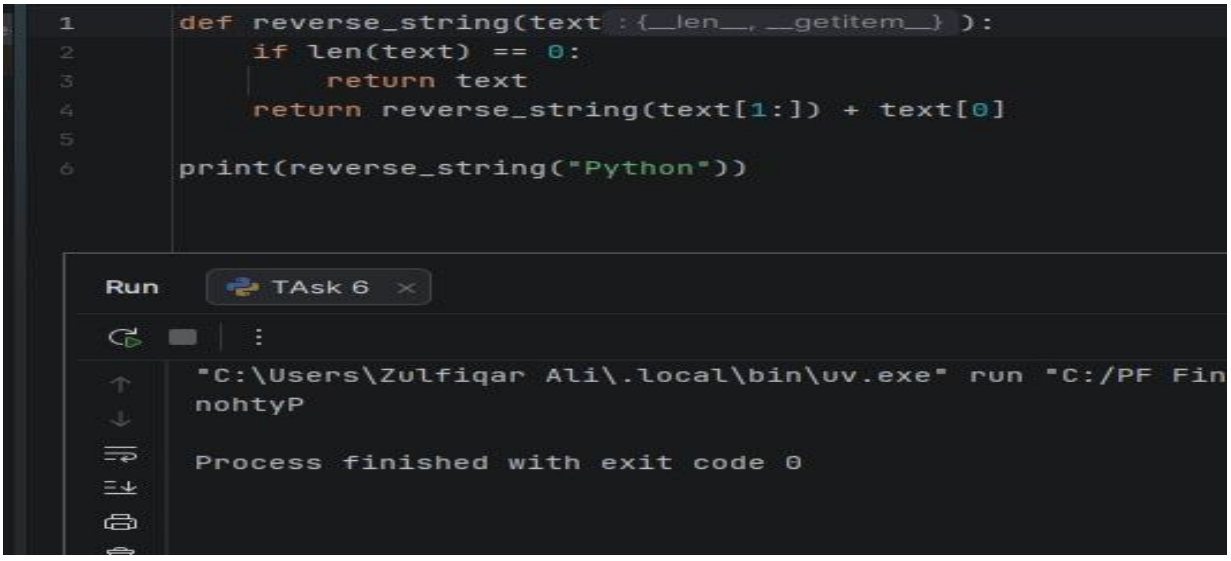
Write a Python program to calculate the power of a number using recursion.

**Result:**


## Task 6:

Write a Python program to reverse a string using recursion.

**Result:**



## Conclusions:

Recursion is a powerful programming technique where a function calls itself to solve a problem. It simplifies solutions for problems that can be divided into smaller subproblems, such as factorials, Fibonacci numbers, string reversal, and mathematical calculations. Proper use of a base

case is essential to prevent infinite recursion and ensure correct program execution.

| University Of Engineering and Technology, Lahore<br>Computer Engineering Department |
|---|

| | |
|---|---|
| **Course Name: Programing Fundamentals** | **Course Code:** CMPE-112L |
| **Assignment Type:** | **Dated: 23 March,2026** |
| **Semester: 1st** | **Session: Spring 2026** |
| **Lab/Project/Assignment #: 11** | **CLOs to be covered: CLO 4** |
| **Lab Title: Exception Handling** | **Teacher Name:** Hafiz Muhammad Mubashar |

## Lab Evaluation:

| **CLO1** | Apply appropriate programming techniques to create executable programs to solve well defined problems | | | | | |
|---|---|---|---|---|---|---|
| | **Level1** | **Level2** | **Level3** | **Level4** | **Level5** | **Level6** |
| (5) | | | | | | |
| | | | | | **Total** | /5 |

## Rubrics for Current Lab Evaluation

| Scale | Marks | Level | Rubric |
|---|---|---|---|
| Excellent | 5 | L1 | Submitted all lab tasks, BONUS task, have good understanding. |
| Very Good | 4 | L2 | Submitted the lab tasks but have good understanding |

| | | | |
|---|---|---|---|
| Good | 3 | L3 | Submitted the lab tasks but have weak understanding. |
| Basic | 2 | L4 | Submitted the lab tasks but have no understanding. |
| Barely Acceptable | 1 | L5 | Submitted only one lab task. |
| Not Acceptable | 0 | L6 | Did not attempt |

# LAB NO 11

## Lab Goals/Objectives:

By reading this manual, students will be able to:

- Understand the concept of Exception Handling in Python.
- Use different exception handling statements to manage runtime errors.
- Develop robust programs by handling exceptions effectively.

## Equipment Required:

Computer system with PyCharm IDE and Python installed on it.

## Theory:

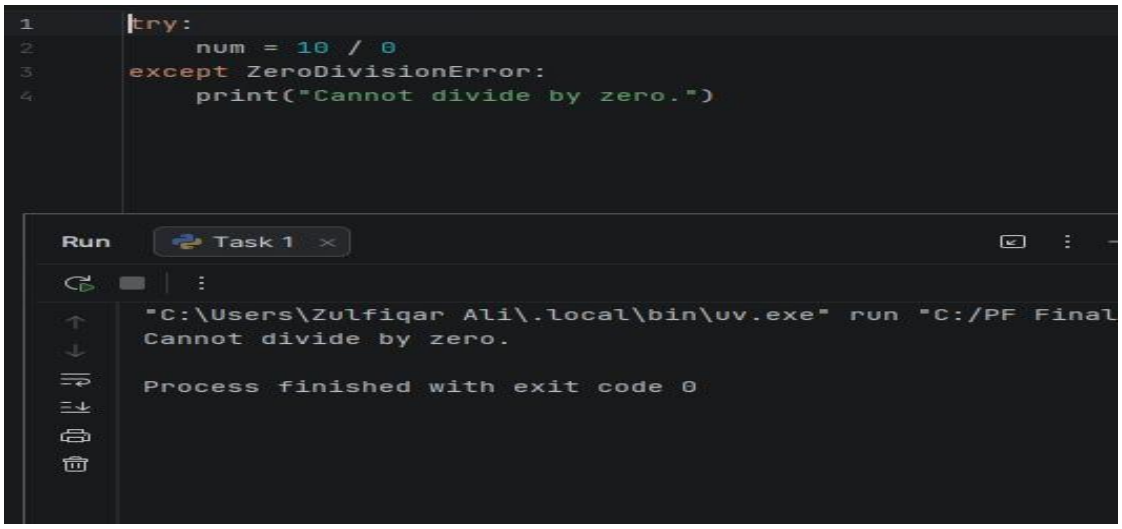
Exception Handling is a mechanism used to handle runtime errors and prevent program crashes. Python provides the `try`, `except`, `else`, and `finally` blocks for handling exceptions. The `try` block contains code that may generate an error. The `except` block handles the error. The `else` block executes when no exception occurs, and the `finally` block executes regardless of whether an exception occurs or not.

**Lab Work:**

### Task 1:

Write a Python program to handle a division-by-zero error using `try` and `except`.

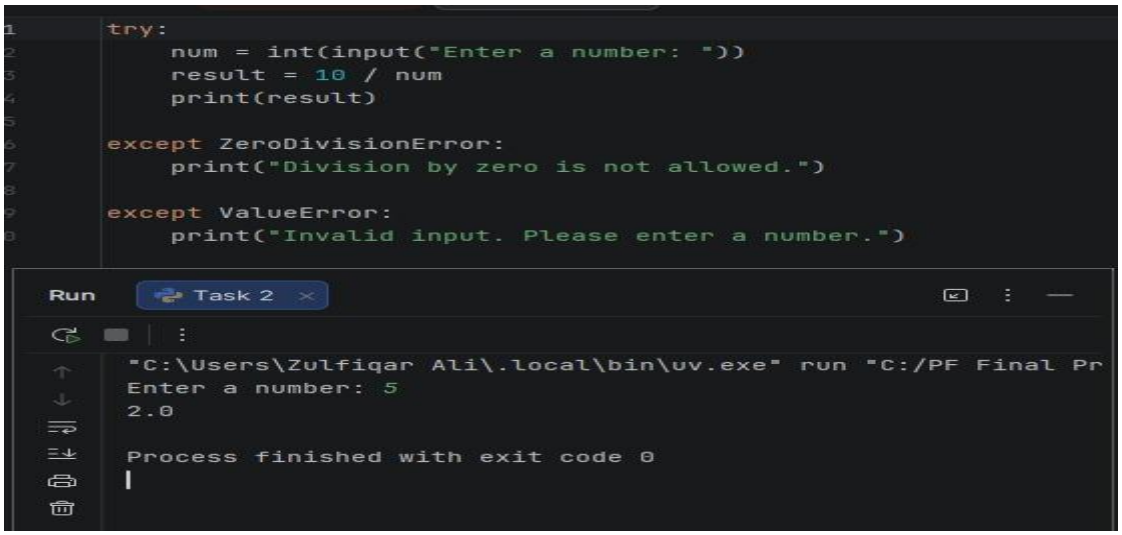
**Result:**




## Task 2:

Write a Python program to handle multiple types of exceptions.

**Result:**




## Task 3:

Write a Python program to demonstrate the use of the `else` block in exception handling.

**Result:**

```
try:
    num = int(input("Enter a number: "))
    print("You entered:", num)

except ValueError:
    print("Invalid input.")

else:
    print("No exception occurred.")
```

```
Run  Task 3
"C:\Users\Zulfiqar Ali\.local\bin\uv.exe" run "C:/PF F
Enter a number: 6
You entered: 6
No exception occurred.

Process finished with exit code 0
```

**Task 4:**

Write a Python program to demonstrate the use of the `finally` block.

**Result:**

```
try:
    num = 10 / 2
    print("Result =", num)

except ZeroDivisionError:
    print("Error occurred.")

finally:
    print("This block always executes.")
```

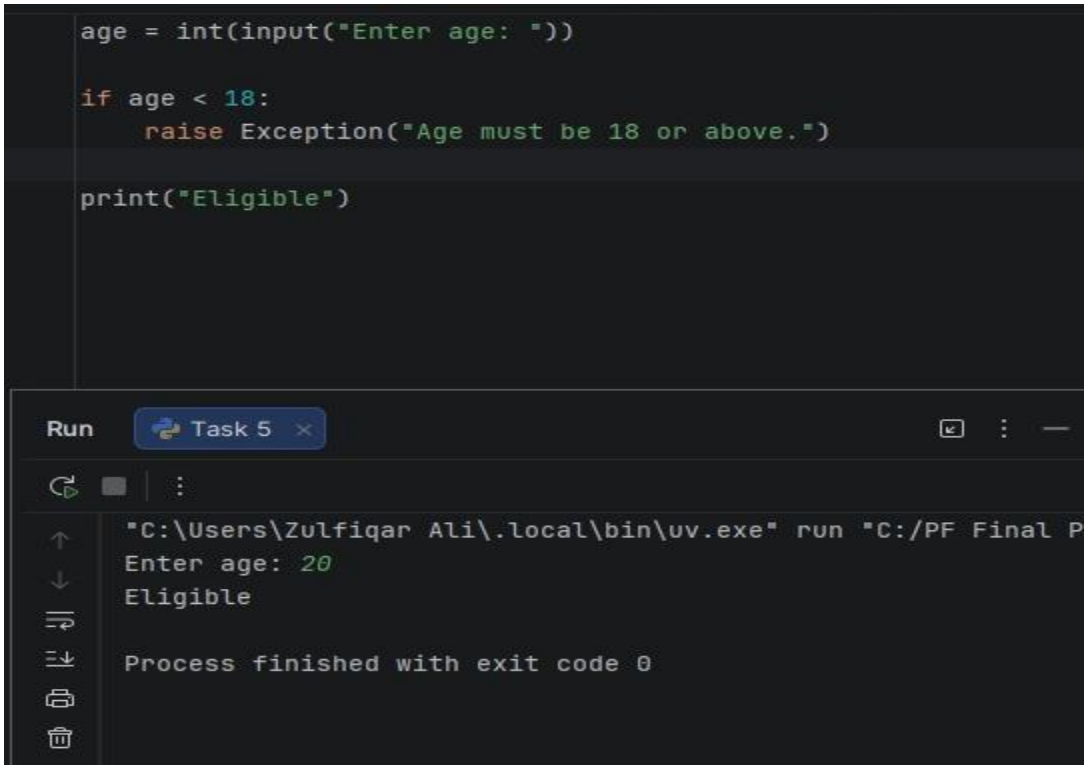
```
Run  Task 4
"C:\Users\Zulfiqar Ali\.local\bin\uv.exe" run "C:/PF Fi
Result = 5.0
This block always executes.

Process finished with exit code 0
```

**Task 5:**

Write a Python program to raise a custom exception using the `raise` keyword.

**Result:**




**Conclusions:**

Exception Handling allows Python programs to detect and manage runtime errors gracefully. Using `try`, `except`, `else`, `finally`, and `raise` helps create reliable and user-friendly programs by preventing unexpected crashes and handling errors effectively.

| University Of Engineering and Technology, Lahore<br>Computer Engineering Department |
|---|

| | |
|---|---|
| **Course Name: Programing Fundamentals** | **Course Code:** CMPE-112L |
| **Assignment Type:** | **Dated: 23 March,2026** |
| **Semester: 1st** | **Session: Spring 2026** |
| **Lab/Project/Assignment #: 12** | **CLOs to be covered: CLO 2** |
| **Lab Title: Lambda Functions** | **Teacher Name:** Hafiz Muhammad Mubashar |

**Lab Evaluation:**

| **CLO1** | Apply appropriate programming techniques to create executable programs to solve well defined problems | | | | | |
|---|---|---|---|---|---|---|
| | **Level1** | **Level2** | **Level3** | **Level4** | **Level5** | **Level6** |
| (5) | | | | | | |
| | | | | | **Total** | /5 |

**Rubrics for Current Lab Evaluation**

| Scale | Marks | Level | Rubric |
|---|---|---|---|
| Excellent | 5 | L1 | Submitted all lab tasks, BONUS task, have good understanding. |
| Very Good | 4 | L2 | Submitted the lab tasks but have good understanding |

| | | | |
|---|---|---|---|
| Good | 3 | L3 | Submitted the lab tasks but have weak understanding. |
| Basic | 2 | L4 | Submitted the lab tasks but have no understanding. |
| Barely Acceptable | 1 | L5 | Submitted only one lab task. |
| Not Acceptable | 0 | L6 | Did not attempt |

# LAB NO 12

## Lab Goals/Objectives:

By reading this manual, students will be able to:

- Understand the concept and syntax of Lambda Functions in Python.
- Use Lambda Functions to perform simple operations efficiently.
- Apply Lambda Functions with built-in functions such as `map()`, `filter()`, and `sorted()`.

## Equipment Required:

Computer system with PyCharm IDE and Python installed on it

## Theory:

A Lambda Function is a small anonymous function in Python that can have any number of arguments but only one expression. Lambda functions are defined using the `lambda` keyword and are commonly used for short, simple operations where creating a regular function is unnecessary.

## Lab Work:

### Task 1:

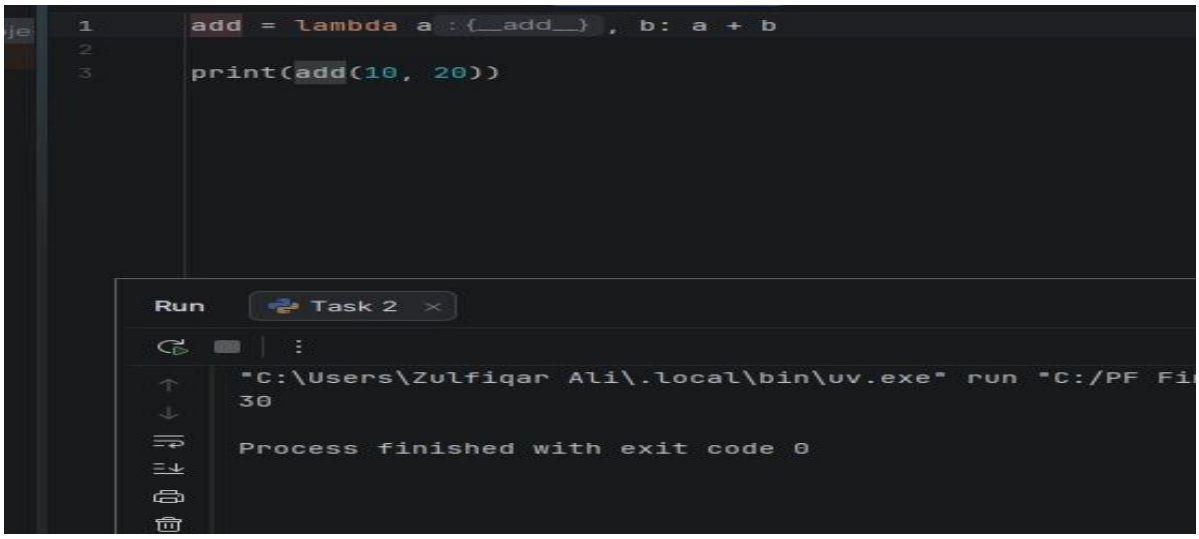
Write a Python program to create a lambda function that returns the square of a number.

**Result:**


**Task 2:**

Write a Python program to create a lambda function that adds two numbers.

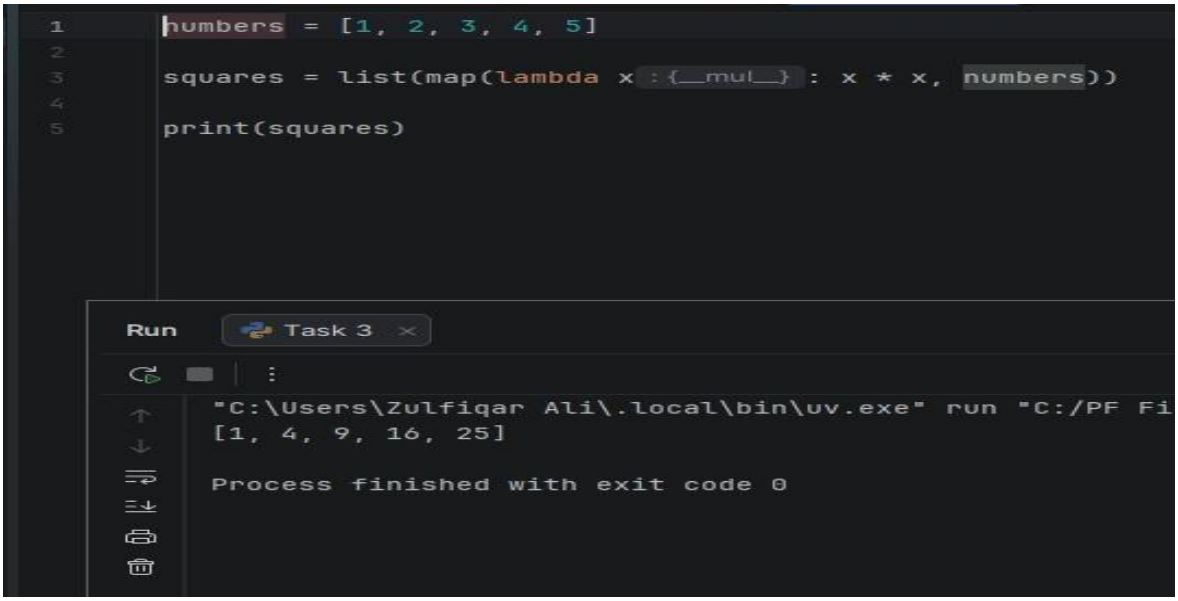
**Result:**




**Task 3:**

Write a Python program to use a lambda function with `map()` to calculate the squares of list elements.

**Result:**




## Task 4:

Write a Python program to use a lambda function with `filter()` to find even numbers from a list.

**Result:**

```
numbers = [1, 2, 3, 4, 5, 6]

even_numbers = list(filter(lambda x :{__mod__} : x % 2 == 0, numbers))

print(even_numbers)
```

```
Run  Task 4
"C:\Users\Zulfiqar Ali\.local\bin\uv.exe" run "C:/PF Final Pr
[2, 4, 6]

Process finished with exit code 0
```

**Task 5:**

Write a Python program to sort a list of tuples using a lambda function.

**Result:**

```
students = [("Ali", 85), ("Ahmed", 75), ("Sara", 90)]

sorted_students = sorted(students, key=lambda x :{__getitem__} : x[

print(sorted_students)
```

```
Run  Task 5
"C:\Users\Zulfiqar Ali\.local\bin\uv.exe" run "C:/PF Final Pr
[('Ahmed', 75), ('Ali', 85), ('Sara', 90)]

Process finished with exit code 0
```

## Conclusions:

Lambda Functions are anonymous functions used for short and simple operations in Python. They make code concise and are commonly used with functions such as `map()`, `filter()`, and `sorted()`. Lambda functions help improve code readability when a full function definition is not required.

| University Of Engineering and Technology, Lahore<br>Computer Engineering Department |
|---|

| | |
|---|---|
| **Course Name: Programing Fundamentals** | **Course Code:** CMPE-112L |
| **Assignment Type:** | **Dated: 23 March,2026** |
| **Semester: 1st** | **Session: Spring 2026** |
| **Lab/Project/Assignment #: 13** | **CLOs to be covered: CLO 2** |
| **Lab Title: Map, Filter and Reduce** | **Teacher Name:** Hafiz Muhammad Mubashar |

**Lab Evaluation:**

| **CLO1** | Apply appropriate programming techniques to create executable programs to solve well defined problems | | | | | |
|---|---|---|---|---|---|---|
| | **Level1** | **Level2** | **Level3** | **Level4** | **Level5** | **Level6** |
| (5) | | | | | | |
| | | | | | **Total** | /5 |

**Rubrics for Current Lab Evaluation**

| Scale | Marks | Level | Rubric |
|---|---|---|---|
| Excellent | 5 | L1 | Submitted all lab tasks, BONUS task, have good understanding. |
| Very Good | 4 | L2 | Submitted the lab tasks but have good understanding |

| | | | |
|---|---|---|---|
| Good | 3 | L3 | Submitted the lab tasks but have weak understanding. |
| Basic | 2 | L4 | Submitted the lab tasks but have no understanding. |
| Barely Acceptable | 1 | L5 | Submitted only one lab task. |
| Not Acceptable | 0 | L6 | Did not attempt |

# LAB NO 13

## Lab Goals/Objectives:

By reading this manual, students will be able to:

- Understand the purpose of `map()`, `filter()`, and `reduce()` functions in Python.
- Apply these functions to process and manipulate data efficiently.
- Use functional programming techniques to write concise Python code.

## Equipment Required:

Computer system with PyCharm IDE and Python installed on it.

## Theory:

Python provides powerful built-in functions such as `map()`, `filter()`, and `reduce()` for processing collections of data.

- **map()** applies a function to every item in an iterable and returns the transformed values.
- **filter()** selects only those elements that satisfy a specified condition.
- **reduce()** repeatedly applies a function to the elements of an iterable and reduces them to a single value. The `reduce()` function is available in the `functools` module.

These functions are commonly used with Lambda Functions to write shorter and more efficient code.

## Lab Work:

**Task 1:**

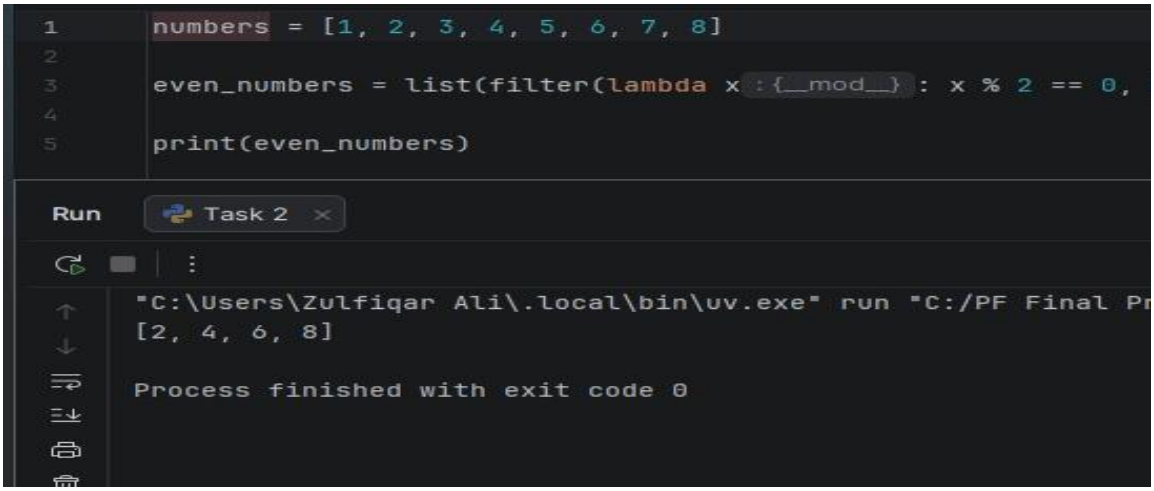
Write a Python program to use `map()` to find the squares of numbers in a list.

**Result:**


**Task 2:**

Write a Python program to use `filter()` to display only even numbers from a list.

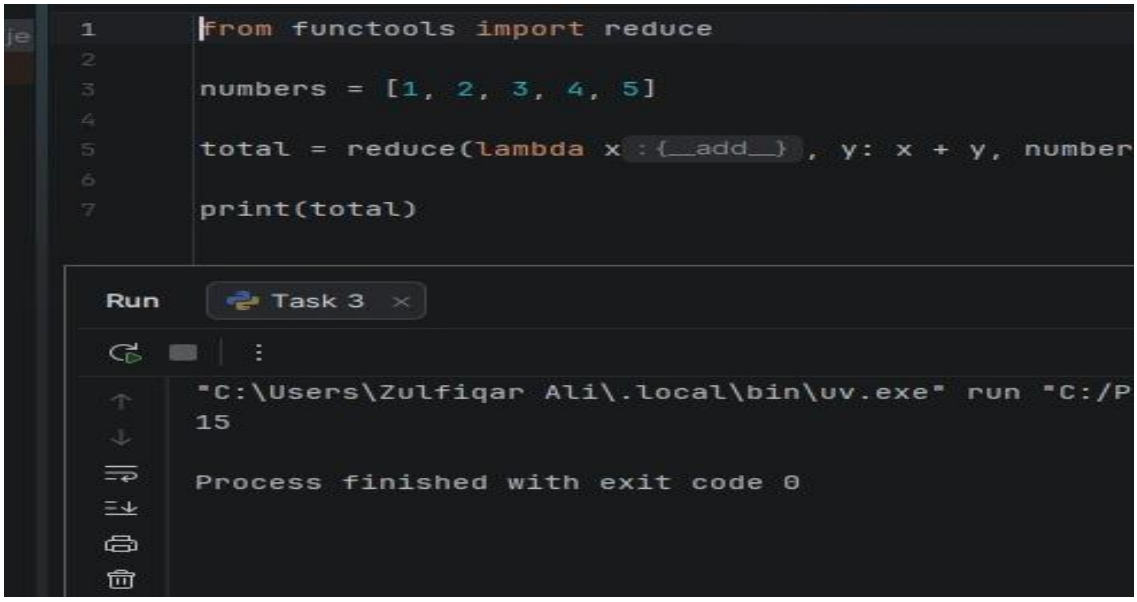
**Result:**




**Task 3:**

Write a Python program to use `reduce()` to find the sum of all elements in a list.

**Result:**




**Task 4:**

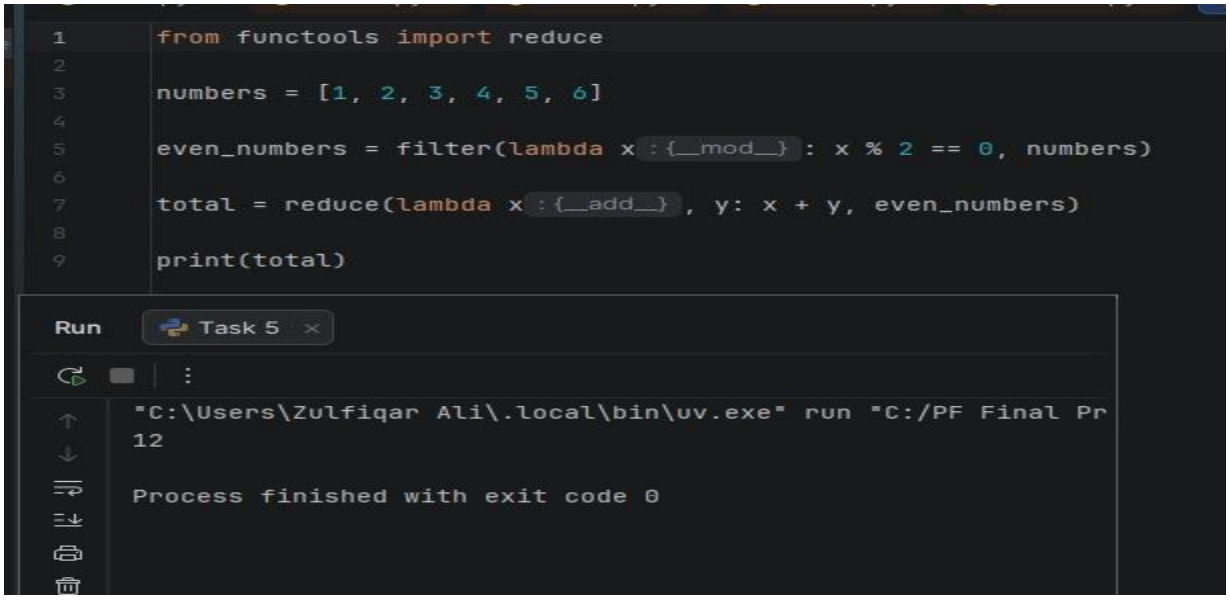
Write a Python program to convert all names in a list to uppercase using `map()`.

**Result:**


**Task 5:**

Write a Python program to find the sum of all even numbers in a list using `filter()` and `reduce()`.

**Result:**




## Conclusions:

`map()`, `filter()`, and `reduce()` are important functional programming tools in Python. `map()` transforms data, `filter()` selects data based on conditions, and `reduce()` combines data into a single result. These functions help write concise, efficient, and readable programs for data processing tasks.

| University Of Engineering and Technology, Lahore<br>Computer Engineering Department |
|---|

| | |
|---|---|
| **Course Name: Programing Fundamentals** | **Course Code:** CMPE-112L |
| **Assignment Type:** | **Dated: 23 March,2026** |
| **Semester: 1st** | **Session: Spring 2026** |
| **Lab/Project/Assignment #: 14** | **CLOs to be covered: CLO 2** |
| **Lab Title: OS Library, if __name__ == "__main__", is vs ==** | **Teacher Name:** Hafiz Muhammad Mubashar |

**Lab Evaluation:**

| **CLO1** | Apply appropriate programming techniques to create executable programs to solve well defined problems | | | | | |
|---|---|---|---|---|---|---|
| | **Level1** | **Level2** | **Level3** | **Level4** | **Level5** | **Level6** |
| (5) | | | | | | |
| | | | | | **Total** | /5 |

**Rubrics for Current Lab Evaluation**

| Scale | Marks | Level | Rubric |
|---|---|---|---|
| Excellent | 5 | L1 | Submitted all lab tasks, BONUS task, have good understanding. |
| Very Good | 4 | L2 | Submitted the lab tasks but have good understanding |

| | | | |
|---|---|---|---|
| Good | 3 | L3 | Submitted the lab tasks but have weak understanding. |
| Basic | 2 | L4 | Submitted the lab tasks but have no understanding. |
| Barely Acceptable | 1 | L5 | Submitted only one lab task. |
| Not Acceptable | 0 | L6 | Did not attempt |

# LAB NO 14

## Lab Goals/Objectives:

By reading this manual, students will be able to:

- Understand the use of the OS Library in Python.
- Learn the purpose of `if __name__ == "__main__"` in Python programs.
- Differentiate between the `is` and == operators.

## Equipment Required:

Computer system with PyCharm IDE and Python installed on it.

## Theory:

### OS Library

The **OS Library** provides functions for interacting with the operating system. It can be used to get the current working directory, create folders, rename files, and perform other system-related tasks.

### if **name** == "**main**"

Every Python file has a special built-in variable called `__name__`. When a Python file is run directly, `__name__` is set to `"__main__"`. This allows certain code to run only when the file is executed directly and not when it is imported as a module.

### is vs ==

- `==` compares the **values** of two objects.
- `is` compares the **identity (memory location)** of two objects.

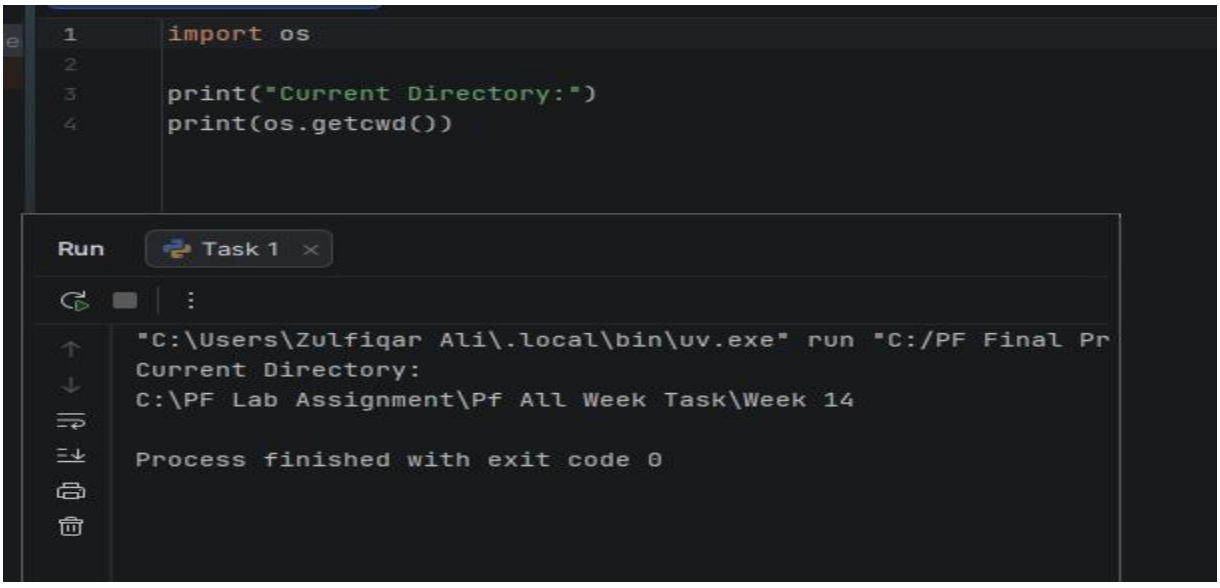
Use `==` when checking whether values are equal and `is` when checking whether two variables refer to the same object.

**Lab Work:**

**Task 1:**

Write a Python program to display the current working directory using the OS Library.

**Result:**




**Task 2:**

Write a Python program to demonstrate the use of `if __name__ == "__main__"`.

**Result:**

```python
def greet():
    print("Welcome to Python")

if __name__ == "__main__":
    greet()
```

```
Run  Task 2
"C:\Users\Zulfiqar Ali\.local\bin\uv.exe" run "C:/PF Final Pr
Welcome to Python

Process finished with exit code 0
```

**Task 3:**

Write a Python program to compare two values using the == operator.

**Result:**

```python
a = 10
b = 10

print(a == b)
```

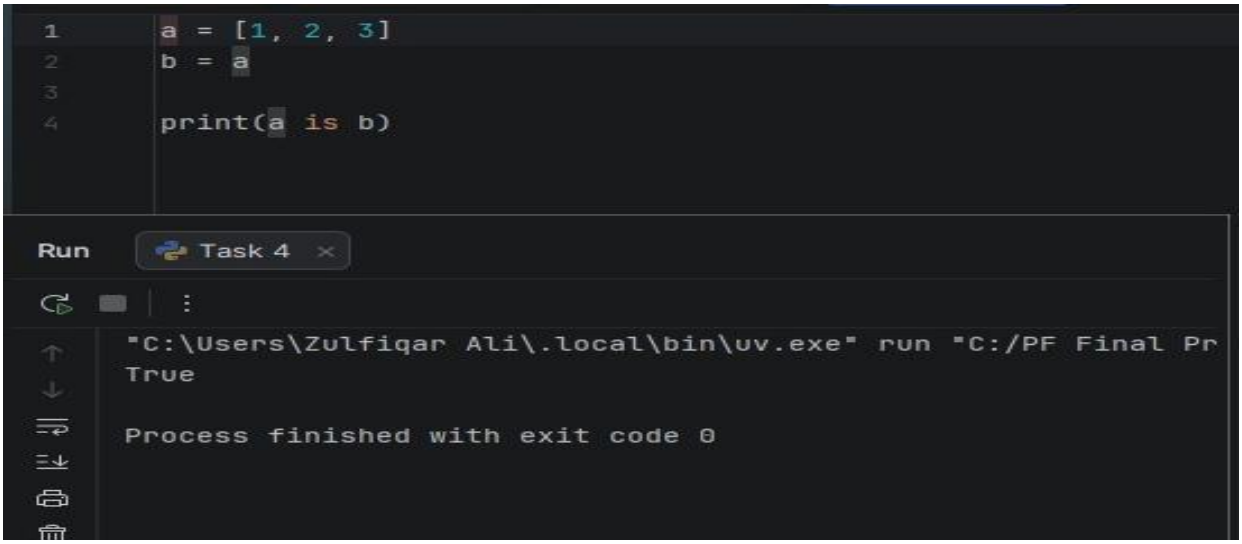
```
Run  Task 3
"C:\Users\Zulfiqar Ali\.local\bin\uv.exe" run "C:/PF Final P
True

Process finished with exit code 0
```

**Task 4:**

Write a Python program to compare two variables using the `is` operator.

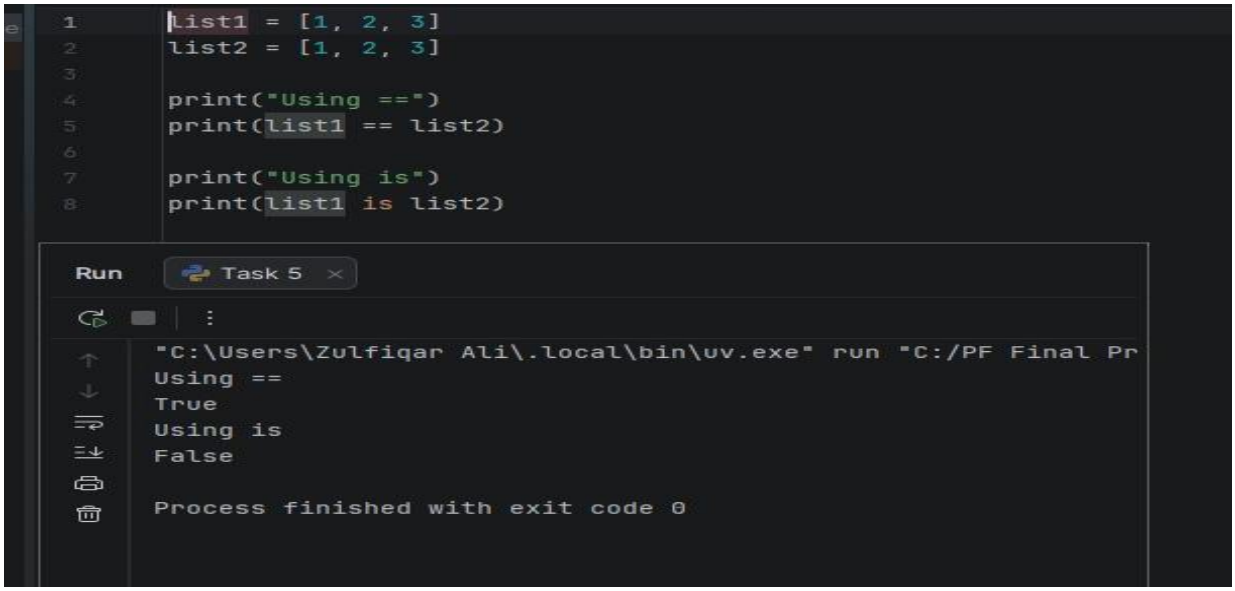
**Result:**




## Task 5:

Write a Python program to demonstrate the difference between `is` and ==.

**Result:**



**Conclusions:**

The OS Library enables interaction with the operating system, `if __name__ == "__main__"` controls the execution of code when a file is run directly, and the `is` and `==` operators are used for identity and value comparison respectively. Understanding these concepts is essential for writing organized and efficient Python programs.

**University Of Engineering and Technology, Lahore**

**Computer Engineering Department**

| | |
|---|---|
| **Course Name: Programing Fundamentals** | **Course Code:** CMPE-112L |
| **Assignment Type:** | **Dated: 07 June,2026** |
| **Semester: 1st** | **Session: Spring 2026** |
| **Lab/Project/Assignment #:** 15 | **CLOs to be covered: CLO 3** |
| **Lab Title: GUI using QT Designer** | **Teacher Name:** Hafiz Muhammad Mubashar |

**Lab Evaluation:**

| **CLO1** | Apply appropriate programming techniques to create executable programs to solve well defined problems | | | | | |
|---|---|---|---|---|---|---|
| | **Level1** | **Level2** | **Level3** | **Level4** | **Level5** | **Level6** |
| (5) | | | | | | |
| | | | | | **Total** | /5 |

**Rubrics for Current Lab Evaluation**

| Scale | Marks | Level | Rubric |
|---|---|---|---|
| Excellent | 5 | L1 | Submitted all lab tasks, BONUS task, have good understanding. |
| Very Good | 4 | L2 | Submitted the lab tasks but have good understanding |
| Good | 3 | L3 | Submitted the lab tasks but have weak understanding. |
| Basic | 2 | L4 | Submitted the lab tasks but have no understanding. |
| Barely Acceptable | 1 | L5 | Submitted only one lab task. |
| Not Acceptable | 0 | L6 | Did not attempt |

# LAB NO 15

## Lab Goals/Objectives:

- Learn the basics of Qt Designer.
- Design a graphical user interface (GUI) without writing code.
- Create a professional login page using Qt widgets.
- Understand widget properties, layouts, fonts, and styling.

**Equipment Required:**

- Computer/Laptop
- Python
- Qt Designer
- PyQt5/PySide6 (installed)
- Windows Operating System

**Theory:**

Qt Designer is a visual tool used to design graphical user interfaces for Qt applications. It allows developers to drag and drop widgets such as labels, buttons, text boxes, and images onto a form without manually writing the interface code.

The designed interface is saved as a **.ui** file, which can later be converted into Python code or loaded directly into a PyQt application. Qt Designer speeds up GUI development and separates interface design from program logic.

Common widgets used in this lab include:

- QLabel
- QTextEdit (used as input fields)
- QPushButton
- QDialog

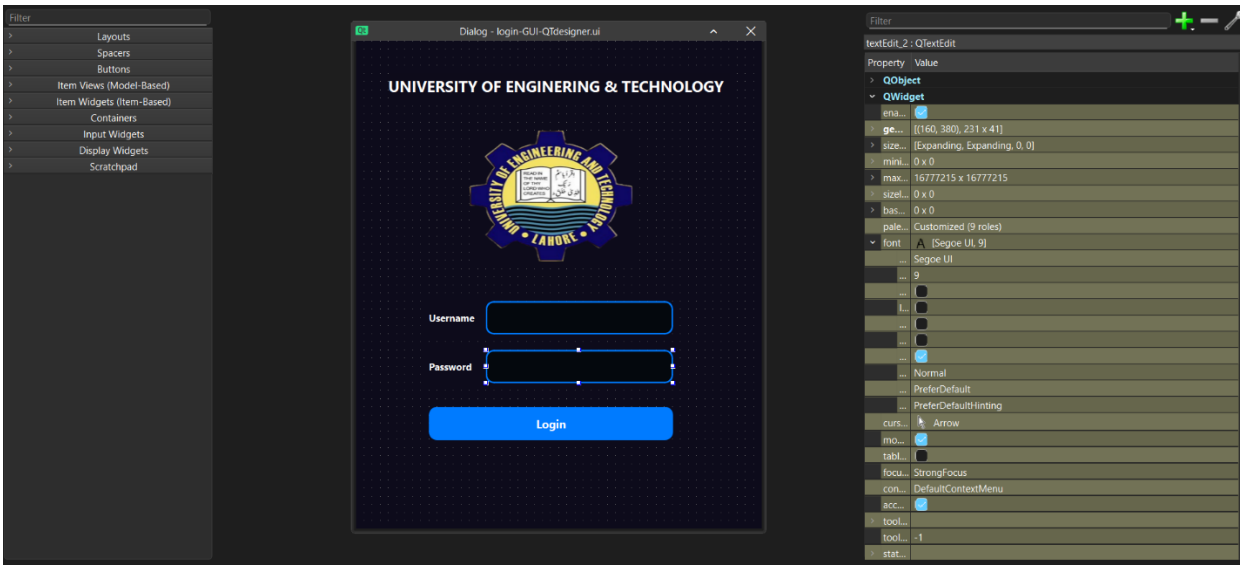
Style sheets were also used to customize colors, fonts, and button appearance.

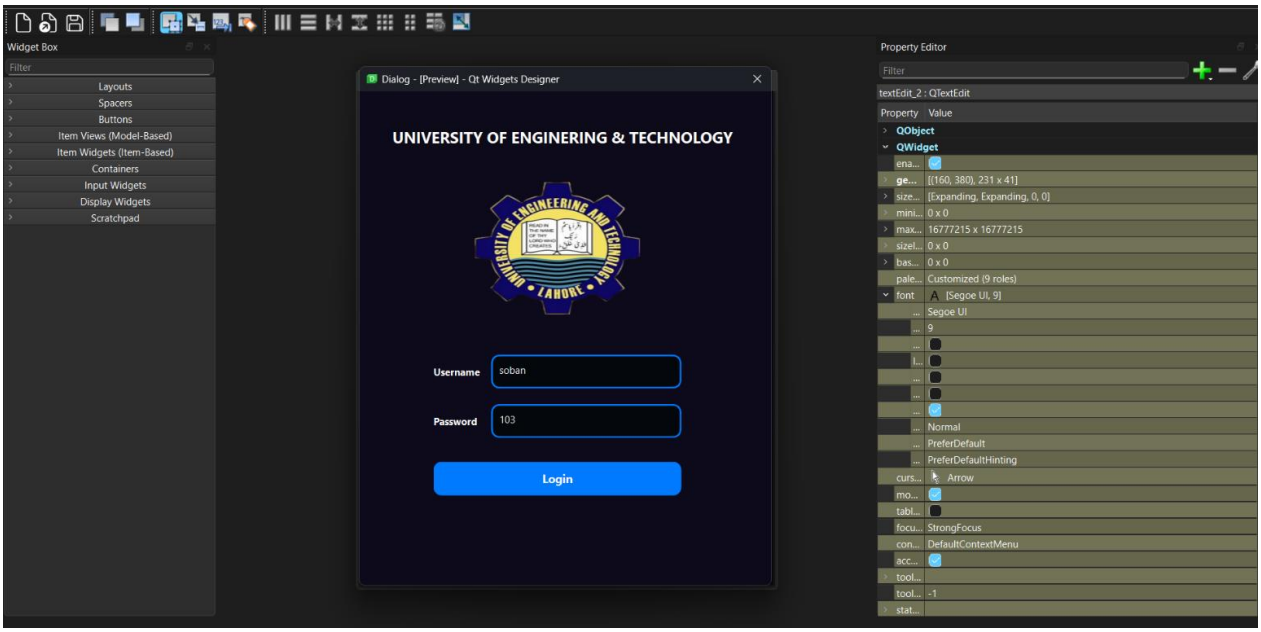
**Lab Work:**

**Task 1:**

Design a University of Engineering and Technology (UET) Login Page using Qt Designer

**Result:**


Filter
Layouts
Spacers
Buttons
Item Views (Model-Based)
Item Widgets (Item-Based)
Containers
Input Widgets
Display Widgets
Scratchpad
Dialog - login-GUI-QTdesigner.ui
UNIVERSITY OF ENGINERING & TECHNOLOGY
LAHORE
Username
Password
Login
textEdit_2 : QTextEdit
Property
Value
QObject
QWidget


Widget Box
Filter
Layouts
Spacers
Buttons
Item Views (Model-Based)
Item Widgets (Item-Based)
Containers
Input Widgets
Display Widgets
Scratchpad
Dialog - [Preview] - Qt Widgets Designer
UNIVERSITY OF ENGINERING & TECHNOLOGY
LAHORE
Username
soban
Password
103
Login
Property Editor
textEdit_2 : QTextEdit
Property
Value
QObject
QWidget

## Conclusions:

In this lab, we learned how to use Qt Designer to create a graphical user interface without writing the interface manually in Python. We gained practical experience with different Qt widgets, their properties, and styling techniques. The login page was successfully designed with a professional appearance, demonstrating the basics of GUI development using Qt Designer.